IF4031 Arsitektur Aplikasi Terdistribusi

Tugas Besar – Milestone 1: Sistem Terdistribusi Dasar

ARGUS

Aggregated Real-time Geohazard Unified System

Gateway koordinasi kebencanaan BNPB yang menyatukan data BMKG dan PVMBG

Nama kelompok	OrangSuksesAamiin
Anggota	13523128 - Andi Farhan Hidayat
	13523164 - Muhammad Rizain Firdaus
	13523152 - Muhammad Kinan Arkansyaddad
Mata kuliah	IF4031 Arsitektur Aplikasi Terdistribusi
Institusi	Institut Teknologi Bandung
Semester	7, 2026/2027

Berkas ini dikumpulkan sebagai `IF4031_M1_OrangSuksesAamiin.pdf`.

Daftar Isi

1 Bagian Umum **1**
- 1.1 Tabel kontribusi . . . 1
- 1.2 Deskripsi umum sistem . . . 1
- 1.3 Diagram arsitektur . . . 2
- 1.4 Teknologi terpilih . . . 4
- 1.5 Asumsi . . . 4
- 1.6 Petunjuk menjalankan . . . 5
- 1.7 Istilah penting . . . 5

2 Problem 1 – Arsitektur dan Interoperabilitas Data **7**
- 2.1 Rancangan solusi dan mekanisme . . . 7
  - 2.1.1 Aturan pemetaan . . . 8
  - 2.1.2 Jenis perubahan skema yang didukung dan tidak . . . 8
- 2.2 Alasan keputusan dan trade-off . . . 8
- 2.3 Batasan yang disadari . . . 10
- 2.4 Bukti penyelesaian . . . 10
- 2.5 Jawaban . . . 12

3 Problem 2 – Concurrency, Availability, dan Graceful Degradation **13**
- 3.1 Rancangan solusi dan mekanisme . . . 13
  - 3.1.1 Pencegahan tiga kegagalan . . . 14
- 3.2 Alasan keputusan dan trade-off . . . 14
- 3.3 Batasan yang disadari . . . 15
- 3.4 Konfigurasi dan tooling load test . . . 15
- 3.5 Bukti penyelesaian . . . 15
- 3.6 Jawaban . . . 17

4 Problem 3 – Autentikasi dan Kepercayaan antar-Service **18**
- 4.1 Rancangan solusi dan mekanisme . . . 18
  - 4.1.1 Klien downstream: Media, Tim Lapangan, BNPB Pusat . . . 18
- 4.2 Alasan keputusan dan trade-off . . . 19
- 4.3 Batasan yang disadari . . . 20
- 4.4 Bukti penyelesaian . . . 20
- 4.5 Jawaban . . . 22

5 Problem 4 – Arsitektur Microservice dan Flexible Storage **23**
- 5.1 Rancangan solusi dan mekanisme . . . 23
  - 5.1.1 Pembagian service . . . 23
  - 5.1.2 Aturan kepemilikan penyimpanan . . . 24
  - 5.1.3 Penyimpanan yang fleksibel: PostgreSQL dengan JSONB . . . 24
  - 5.1.4 Isolasi jaringan sebagai bukti kepemilikan . . . 24
- 5.2 Alasan keputusan dan trade-off . . . 24

5.2.1 Batas service: mengapa dipecah seperti ini . . . 25
5.2.2 Pilihan penyimpanan . . . 25
5.3 Batasan yang disadari . . . 26
5.4 Bukti penyelesaian . . . 26
5.5 Jawaban . . . 28

6 Problem 5 – Fan-out Event Distribution (Pub/Sub) 29
6.1 Rancangan solusi dan mekanisme . . . 29
6.1.1 Sisi producer: transactional outbox . . . 29
6.1.2 Sisi consumer . . . 30
6.1.3 Semantik pengiriman: at-least-once . . . 30
6.2 Alasan keputusan dan trade-off . . . 30
6.3 Batasan yang disadari . . . 31
6.4 Bukti penyelesaian . . . 32
6.5 Jawaban . . . 33

A Penelusuran Klaim ke Bukti 34
A.1 Ketentuan implementasi lintas problem . . . 34
A.2 Matriks kriteria, perintah, dan bukti . . . 34
A.3 Perintah untuk pemeriksaan penuh . . . 35

B Lampiran Surat Pernyataan Penggunaan AI 36

1 Bagian Umum

1.1 Tabel kontribusi

Tabel berikut memuat pembagian kerja anggota. Pengerjaan dibagi dalam tiga tahap: fondasi dan Problem 1, kemudian Problem 2 dan 3, dan terakhir Problem 4 dan 5 beserta integrasinya.

Anggota	Perancangan	Implementasi	Penulisan laporan
13523128 - Andi Farhan Hidayat	Kontrak antarkomponen, arsitektur Aggregator, kebijakan skema	Fondasi (`pkg/obs`, Compose, Makefile, CI), *mock* BMKG dan PVMBG, Aggregator, Problem 1, ketahanan proses ingest, adapter PostgreSQL dan Kafka, *outbox* dan *relay*, *consumer*	Bagian umum, Problem 1, deskripsi *mock*
13523164 - Muhammad Rizain Firdaus	Autentikasi JWT dengan tiga peran, kebijakan *field* Ringkasan dan Mentah, *rate limiter*, penanda `stale`	`auth-service`, `client-api`, skrip demo dan *load test* Problem 2 dan 3	Problem 2, Problem 3
13523152 - Muhammad Kinan Arkansyaddad	Perbandingan pilihan penyimpanan (PostgreSQL dengan JSONB) dan pilihan distribusi *event* (Kafka dengan *outbox*), semantik pengiriman, serta batasan yang disadari	Skrip demo Problem 4 dan 5 (*rebuild* satu *service*, *field* baru tanpa migrasi, *consumer* baru) dan pengujian pemulihan *consumer*	Problem 4, Problem 5

Tabel 1.1: Kontribusi anggota.

1.2 Deskripsi umum sistem

ARGUS (*Aggregated Real-time Geohazard Unified System*) adalah *gateway* milik BNPB yang menyatukan data kebencanaan dari BMKG dan PVMBG. Sistem ini menjalankan lima fungsi:

1. Mengambil data dari dua lembaga secara berkala dengan *polling*: BMKG untuk gempa dan tsunami, PVMBG untuk gunung api.
2. Memetakan dua format sumber yang berbeda ke satu format milik BNPB, yaitu *HazardEvent*.
3. Menyimpan hasilnya di satu *storage*, yaitu Canonical Store, yang hanya dikelola oleh satu *service*.
4. Mem-*publish* setiap data baru ke *message broker* agar bisa di-*subscribe* oleh pihak yang berkepentingan.
5. Menyajikan data kepada tiga jenis klien (Media, Tim Lapangan, BNPB Pusat) dengan hak akses yang berbeda.

Lingkup Milestone 1 mengikuti spesifikasi tugas: dua lembaga sumber (BMKG dan PVMBG), dua jenis bencana (seismik dan vulkanik), BNPB sebagai simpul koordinasi, dan semua klien bersifat *read-only*. BMKG dan PVMBG berupa *mock* yang dibuat sendiri oleh kelompok sesuai kontrak pada spesifikasi, tanpa mengubahnya.

Status penyelesaian

Tabel berikut merangkum hasil demo tiap problem. Bukti P1 dan sisi ingest P2 direkam pada 7 Oktober 2026, sedangkan bukti sisi layanan P2, P3, P4, dan P5 pada 8 Oktober 2026. Demo P2 dan P3 dijalankan sekali setelah `make reset`. Demo P4 dan P5 dijalankan setelah reset, lalu diulang sekali tanpa reset, dan keduanya lulus pada kedua percobaan.

Problem	Hasil yang terbukti	Status
P1 Interoperabilitas data	Pemetaan sesuai aturan, skema PVMBG berubah saat sistem berjalan tanpa *restart*, *field* baru masuk ke `attributes`, dan perubahan yang tidak didukung dikarantina.	**Lulus**
P2 Konkurensi dan ketersediaan	p95 permintaan BMKG-only di bawah 300 ms saat PVMBG diberi *delay* 3 detik, *load test* 50 koneksi selama 60 detik, outage PVMBG tertangani, dan sistem pulih tanpa *restart*.	**Lulus**
P3 Autentikasi dan kepercayaan	Kredensial antarlembaga saling menolak, Media hanya menerima *field* Ringkasan, *token* Tim Lapangan kedaluwarsa lalu di-*refresh*, dan tidak ada kredensial yang ter-*commit*.	**Lulus**
P4 Microservice dan *storage*	Satu *container* (`pvmbg-mock`) dihentikan, di-*rebuild*, dan dijalankan lagi tanpa menyentuh *container* lain. *Field* baru masuk ke JSONB tanpa migrasi, *record* lama tetap terbaca, dan Canonical Store hanya terjangkau oleh Aggregator.	**Lulus**
P5 Pub/sub	*Event* dipublikasikan lewat Kafka ke dua *consumer* independen. Satu *consumer* dimatikan lalu menyusul *event* yang terlewat, dan *consumer* ketiga ditambahkan saat sistem berjalan tanpa mengubah *producer*.	**Lulus**

Tabel 1.2: Status per problem pada saat laporan ini ditulis.

1.3 Diagram arsitektur

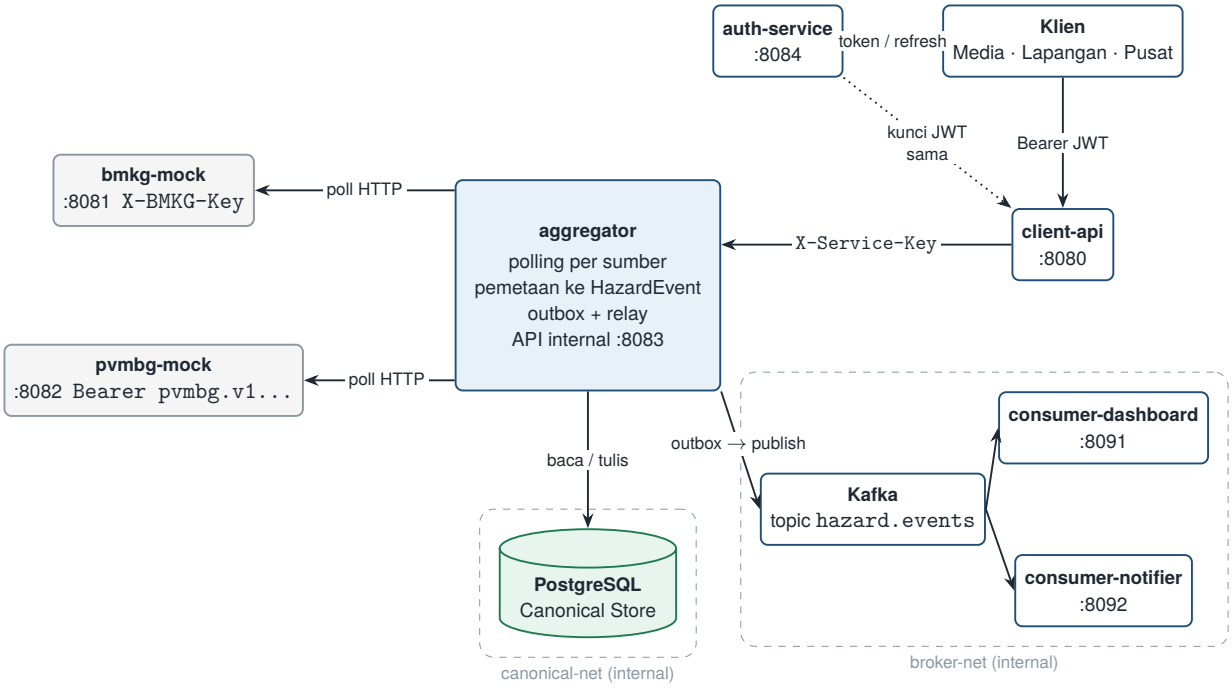
Gambar 1.1 menunjukkan seluruh *service*, protokol komunikasi, kepemilikan *storage*, dan batas jaringan.

Penjelasan

Alur masuk. `aggregator` memanggil `bmkg-mock` dan `pvmbg-mock` lewat HTTP. Kedua *mock* bersifat pasif: mereka hanya menjawab permintaan dan tidak mengirim data sendiri. Setiap sumber punya putaran *polling* sendiri, sehingga PVMBG yang lambat tidak menahan BMKG (Problem 2). Hasil *polling* dipetakan ke *HazardEvent* lalu disimpan.

Penyimpanan. Canonical Store adalah PostgreSQL yang hanya berada di jaringan `canonical-net`. Jaringan itu hanya berisi `aggregator` dan databasenya, sehingga *service* lain secara fisik tidak bisa terhubung ke database tersebut (Problem 4). *Service* lain mengambil data lewat API internal Aggregator.

Alur keluar ke klien. Klien meminta *token* ke `auth-service`, lalu memanggil `client-api` dengan *token* itu. `client-api` memeriksa *token*, menentukan *field* yang boleh dilihat oleh peran ter-



Gambar 1.1: Arsitektur sistem. Garis tegas: panggilan jaringan; garis titik: rahasia penandatangan token yang dipakai bersama. Kotak putus-putus: jaringan Docker yang tidak bisa dijangkau dari luar (`internal`).

sebut, dan meneruskan permintaan ke API internal Aggregator dengan *header* `X-Service-Key`. `client-api` tidak pernah memanggil BMKG atau PVMBG (Problem 2 dan 3).

Alur *event*. Saat Aggregator menyimpan sebuah perubahan, ia menulis satu baris *outbox* dalam transaksi database yang sama. Proses *relay* kemudian mengirim baris itu ke *topic* Kafka `hazard.events`. *Consumer* men-*subscribe topic* tersebut tanpa Aggregator mengetahuinya (Problem 5). Setiap alur membawa *correlation ID* yang sama, dari siklus *polling* sampai ke *consumer*.

Daftar container

Sistem terdiri dari sepuluh *container*, satu di antaranya hanya berupa tugas sekali jalan. Tabel berikut memuat port, jaringan, dan penyimpanan tiap *container*.

Container	Port host	Jaringan	Penyimpanan yang dimiliki
`bmkg-mock`	8081	app-net	tidak ada (data simulasi di memori)
`pvmbg-mock`	8082	app-net	tidak ada (data simulasi di memori)
`aggregator`	127.0.0.1:8083	app, canonical, *broker*	Canonical Store (tabel `hazards`, `outbox`, `poll_cursor`)
`postgres-canonical`	tidak dibuka	canonical-net	volume `canonical-pgdata`
`kafka`	tidak dibuka	broker-net	volume `kafka-data`
`kafka-init`	–	broker-net	job sekali jalan untuk membuat *topic*
`auth-service`	8084	app-net, auth-net	SQLite untuk hash *refresh token* (volume `auth-data`)
`client-api`	8080	app-net	tidak ada (*stateless*)
`consumer-dashboard`	8091	*broker*, app	memori (kumpulan `event_id`)
`consumer-notifier`	8092	*broker*, app	memori (kumpulan `event_id`)

Tabel 1.3: *Container*, jaringan, dan kepemilikan penyimpanan (dari `docker-compose.yml`).

1.4 Teknologi terpilih

Tabel berikut memuat teknologi yang dipakai pada tiap komponen beserta kebutuhan sistem yang mendasarinya.

Komponen	Teknologi	Alasan pemilihan
Semua *service*	Go 1.25	Goroutine yang ringan cocok untuk satu putaran *polling* per sumber. Image kecil dan build cepat, sehingga *rebuild* satu *service* tidak lama (Problem 4). Satu bahasa untuk semua *service* juga mengurangi biaya integrasi di antara tiga anggota.
HTTP dan *log*	`net/http`, `log/slog`	Tidak butuh dependensi tambahan. *Log* berformat JSON membawa `cid` lewat *context*, sesuai syarat *log* terstruktur.
Canonical Store	PostgreSQL 16 dengan kolom JSONB	*Field* kanonik disimpan di kolom tetap, sedangkan *field* yang bisa berubah masuk ke `attributes` (JSONB), jadi *field* baru tidak membutuhkan migrasi (Problem 1 dan 4). Transaksi database juga memungkinkan pola *outbox* (Problem 5).
Message broker	Apache Kafka (mode KRaft, satu *broker*)	*Log* yang tahan lama dengan *consumer group* dan *offset*, sehingga *consumer* yang mati bisa menyusul *event* yang terlewat setelah hidup kembali (Problem 5).
Klien Kafka	`franz-go`	Ditulis murni dalam Go (tanpa cgo), sehingga Dockerfile tetap sederhana.
Autentikasi	JWT HS256 dan *refresh token* opaque	Access *token* berumur pendek dan diverifikasi secara lokal, sehingga jalur baca tidak bergantung pada Auth Service (Problem 2 dan 3).
Penyimpanan *token*	SQLite (`modernc.org/sqlite`)	Hanya hash *refresh token* yang disimpan. Cukup untuk satu *instance* dan tanpa cgo.
Rate limiter	`golang.org/x/time/rate`	Algoritma *token bucket* yang langsung menolak kelebihan beban dengan status 429, bukan memasukkannya ke antrean (Problem 2).
Orkestrasi	Docker Compose dengan empat jaringan	Satu perintah menyalakan seluruh sistem. Jaringan *internal* menjadi bukti isolasi penyimpanan.
Load test	k6	Melaporkan *throughput*, p50/p95/p99, dan error rate, serta mendukung ambang batas lulus.
Kualitas kode	GitHub Actions, `gofmt`, `go vet`, `-race`, gitleaks	Pengujian otomatis dan pencegahan kredensial yang ter-*commit*.

Tabel 1.4: Teknologi per komponen dan alasan pemilihannya.

1.5 Asumsi

1. Respons *mock* berupa JSON array. Parameter `since` bersifat *lebih besar dari* (bukan sama dengan) terhadap `occurred_at`, `issued_at`, atau `reported_at`. *Field* `issued_at` pada TsunamiWarning merupakan tambahan yang diizinkan spesifikasi.
2. Warning tsunami dapat terbit *setelah* gempanya. *Mock* sengaja memberi jeda 2–8 detik. Satu gempa dapat memiliki beberapa warning, dan warning dengan level tertinggi menentukan severity.
3. Pada M1 hanya ada satu *instance* untuk tiap *service*. Optimistic concurrency (`expectedVersion`) sudah disiapkan untuk M2.

4. Koordinat dan nama gunung api berasal dari tabel referensi statis milik BNPB (`config/volcanoes.json`). Koordinatnya hanya pendekatan untuk simulasi, dan gunung "Prikitiw" bersifat fiktif.
5. Seluruh waktu memakai UTC dan dibulatkan ke mikrodetik.
6. Karantina untuk *record* yang tidak dapat dipetakan hanya bersifat diagnostik dan disimpan di memori.
7. *Endpoint* `/admin/*` pada *mock* (termasuk `extra_fields`, `override`, `/admin/generate`, dan `/admin/delay`) adalah tambahan untuk keperluan demo. Kontrak yang ditetapkan spesifikasi tidak diubah.
8. Semua klien hanya membaca. Jalur pelaporan dari lapangan ke BNPB berada di luar lingkup M1.
9. Jaringan Docker dianggap tepercaya sehingga tidak ada TLS di dalamnya. Pada lingkungan produksi, TLS dipasang di ingress.
10. Kredensial pada `.env.example` hanya contoh. Nilai sebenarnya diisi pada `.env` yang tidak pernah di-*commit*.

1.6 Petunjuk menjalankan

Langkah lengkap, daftar variabel konfigurasi, dan cara memicu tiap skenario ada di `README.md` pada repositori. Ringkasannya sebagai berikut.

```
cp .env.example .env            # nilai contoh; .env tidak pernah di-commit
docker compose up --build       # satu perintah untuk seluruh sistem (atau: make up)
make demo-p1                    # lalu demo-p2, demo-p3, demo-isolation, demo-p4, demo-p5, dst.
make reset                      # kembali ke kondisi bersih (hapus container dan volume)
```

Satu-satunya prasyarat adalah Docker dengan Compose v2. Port bawaan: BMKG 8081, PVMBG 8082, Client-Facing API 8080, Auth Service 8084, Aggregator 8083 (hanya *loopback*), serta *consumer* 8091 dan 8092. Semua port dapat diubah lewat `.env`. Setiap demo memeriksa hasilnya sendiri (kode keluar tidak nol jika gagal) dan menyimpan transkripnya di `docs/evidence/`.

1.7 Istilah penting

Istilah berikut sering muncul di laporan ini.

Istilah	Arti dalam laporan ini
HazardEvent	Format tunggal milik BNPB untuk satu kejadian bahaya, dari sumber mana pun.
Canonical Store	Tempat HazardEvent disimpan. Hanya Aggregator yang boleh mengaksesnya langsung.
Ringkasan / Mentah	Dua klasifikasi *field*. *Field* Ringkasan boleh dilihat Media, sedangkan *field* Mentah (koordinat presisi, `attributes`, `source_ref_id`) tidak.
Polling	BNPB menanyakan data ke sumber secara berkala. Sumber tidak mengirim data sendiri.
Outbox	Tabel di database yang menampung *event* sampai berhasil dikirim ke *broker*, sehingga tidak ada *event* yang hilang saat *broker* sedang mati.
At-least-once	Setiap *event* pasti sampai, tetapi bisa sampai lebih dari sekali.
Idempotent	Memproses hal yang sama dua kali menghasilkan keadaan yang sama dengan memprosesnya sekali.
Circuit breaker	Setelah sebuah sumber gagal berulang kali, pemanggilan ke sumber itu dihentikan sementara agar tidak membebani sumber yang sedang bermasalah.
Token bucket	Algoritma *rate limiter*: tiap permintaan mengambil satu *token*, dan permintaan ditolak dengan status 429 jika *token* habis.
Stale	Data tetap ditampilkan, tetapi diberi tanda bahwa sumbernya sedang tidak diperbarui.
Correlation ID (`cid`)	Kode yang sama pada semua *log* dalam satu alur permintaan, dari *service* pertama sampai yang terakhir.

Tabel 1.5: Daftar istilah.

2 Problem 1 – Arsitektur dan Interoperabilitas Data

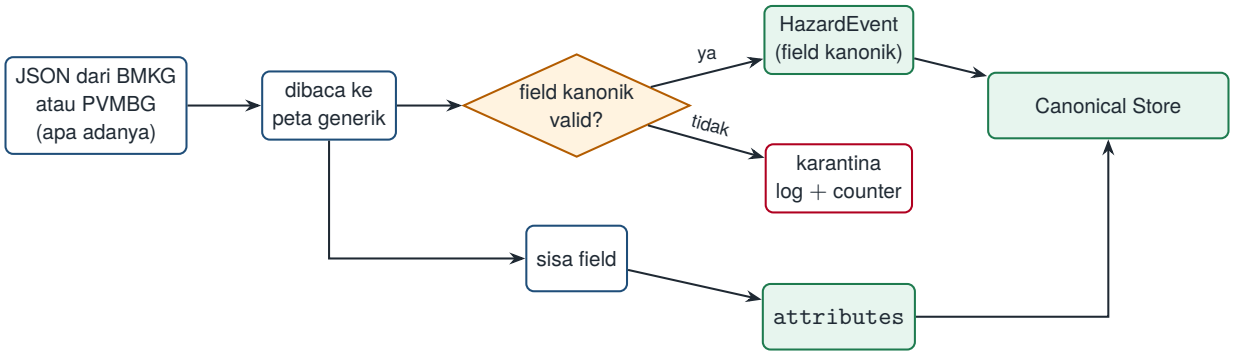
Ringkasan

PVMBG dapat menambah *field* baru kapan saja tanpa memberi tahu siapa pun, sehingga *gateway* tidak boleh menganggap skema sumber tetap. *Field* yang dikenal divalidasi ketat, sedangkan *field* lainnya diteruskan apa adanya ke `attributes`. Hanya perubahan yang mengubah *arti* kontrak (*field* wajib hilang, tipe berubah, nilai enum baru) yang dikarantina dan tidak ditebak. Seluruh perilaku ini dibuktikan saat sistem berjalan, tanpa *restart*.

2.1 Rancangan solusi dan mekanisme

Mekanisme yang dipakai adalah *tolerant reader* dengan kebijakan *additive-only*. Aggregator tidak pernah membaca data sumber ke *struct* yang tetap. Setiap *record* dibaca ke peta generik (`map[string]any`), lalu diproses dengan tiga aturan:

1. *Field* yang punya padanan kanonik dibaca secara eksplisit dan divalidasi (harus ada dan bertipe benar), misalnya `magnitude`, `epicenter_lat`, `occurred_at`, `alert_level`, dan `volcano_id`.
2. Semua *field* lain diteruskan apa adanya ke `attributes`. Ini mencakup *field* kontrak yang memang hanya menjadi atribut (`depth_km`, `eruption_count_24h`) dan *field* yang muncul belakangan (`confidence_level` atau *field* apa pun).
3. *Record* yang melanggar kontrak dengan cara yang tidak didukung tidak ditebak, tetapi dikarantina. *Record* tersebut dicatat di *log* (`mapping_rejected`), dihitung, dan dapat dilihat lewat `GET /internal/quarantine`. Sisa batch tetap diproses dan penanda kemajuan *polling* (*watermark*) tetap maju.



Gambar 2.1: Alur pemetaan. Sisa *field* selalu ikut tersimpan, sedangkan *record* yang melanggar kontrak berhenti di karantina.

Kebijakan untuk *field* yang tidak dikenal adalah **meneruskannya ke `attributes`**. Kebijakan ini diterapkan di satu tempat saja (fungsi `splitExtras`), sehingga berlaku sama untuk semua tipe JSON dan kedua sumber. Demo memeriksanya dengan pembanding independen: isi `attributes` harus *persis sama* dengan himpunan *field* sumber yang tidak punya padanan kanonik. Tidak boleh ada yang hilang dan tidak boleh ada yang dikarang.

Pembaruan terhadap *record* yang sama digabung secara aditif. Jika *record* yang sama da-

tang lagi, nilai yang dikirim sumber saat ini menang, tetapi *field* yang berhenti dikirim tetap tersimpan. Tanpa aturan ini, mengembalikan skema sumber ke bentuk semula akan menimpa *record* terbaru dengan versi yang lebih sedikit isinya dan membuang *field* yang sudah sempat tertangkap. Bug ini sempat muncul saat pengujian dan sekarang dijaga oleh `TestSchemaRollbackDoesNotEraseCapturedFields`.

2.1.1 Aturan pemetaan

Spesifikasi menetapkan cara tiap *field* sumber dipetakan ke *field* HazardEvent. Tabel berikut merangkumnya.

Kanonik	SeismicEvent (BMKG)	VolcanicReport (PVMBG)
`hazard_id`	`bmkg-<event_id>`	`pvmbg-<report_id>`
`source_ref_id`	`event_id`	`report_id`
`hazard_type`	SEISMIC	VOLCANIC
`area_name`	`region_name`	nama dari tabel referensi statis BNPB
`latitude`, `longitude`	`epicenter_lat`, `epicenter_lon`	dari tabel referensi statis
`occurred_at`	`occurred_at`	`reported_at`
`severity`	lihat aturan di bawah	`alert_level`: Normal, Waspada, Siaga, dan Awas menjadi NORMAL, WASPADA, SIAGA, dan AWAS
`attributes`	`magnitude`, `depth_km`, `potential_tsunami`, *field* TsunamiWarning, dan *field* yang tidak dikenal	`eruption_count_24h`, `ash_column_height_m`, `confidence_level` jika ada, dan *field* yang tidak dikenal

Tabel 2.1: Pemetaan ke HazardEvent sesuai kontrak pada spesifikasi.

Severity seismik. Severity bernilai AWAS jika ada warning dengan `threat_level = Awas`. Jika tidak, severity mengikuti `threat_level` dari warning terkait. Tanpa warning, severity NORMAL untuk magnitudo di bawah 5,0, WASPADA untuk 5,0 sampai kurang dari 6,5, dan SIAGA untuk 6,5 ke atas. Jika ada beberapa warning, warning dengan level tertinggi yang dipakai.

Warning tsunami yang datang belakangan. Warning diambil dari *endpoint* terpisah dan sering terbit setelah gempanya tersimpan. Aggregator mengaitkannya lewat `related_event_id`, menghitung ulang severity, lalu menyimpannya sebagai *versi baru* (`version` naik dan *event* `updated` dipublikasikan). Warning yang gempanya belum terlihat akan menunggu (dibatasi TTL) dan menahan *watermark* warning agar diambil ulang, bukan hilang. Karena `hazard_id` bersifat deterministik dan penyimpanan membandingkan isi *record*, *polling* ulang pada jendela *overlap* tidak menaikkan versi.

2.1.2 Jenis perubahan skema yang didukung dan tidak

Karena sumber dapat berubah kapan saja, kami menetapkan secara eksplisit perubahan mana yang ditangani otomatis dan mana yang sengaja tidak.

2.2 Alasan keputusan dan trade-off

Ada beberapa cara menghadapi skema sumber yang berubah. Tabel berikut membandingkannya dengan pilihan kami.

Perubahan pada sumber	Perilaku Aggregator	Didukung?
Field baru bertipe apa pun (angka, teks, objek bersarang, list, `null`)	masuk ke `attributes`; HazardEvent tetap valid	Ya
Field baru pada TsunamiWarning	masuk ke `attributes` dengan awalan `warning_` agar namanya tidak bertabrakan	Ya
Field yang hanya berstatus atribut menghilang	atribut itu tidak ada pada *record* baru	Ya
Field baru lalu hilang lagi (skema dikembalikan)	*record* lama tetap menyimpan nilainya, *record* baru tidak memilikinya	Ya
`volcano_id` tidak ada di tabel referensi	`area_name` berisi id mentah, koordinat `null`, dan ada peringatan di *log*	Ya (*graceful degradation*)
Field wajib hilang atau berganti nama	karantina	**Tidak, sengaja**
Tipe *field* wajib berubah (misalnya `magnitude` menjadi teks)	karantina, tidak dipaksa	**Tidak, sengaja**
Nilai enum baru (misalnya `alert_level = Darurat`)	karantina	**Tidak, sengaja**
Perubahan arti atau satuan (meter menjadi kaki)	tidak terdeteksi	Tidak, di luar jangkauan
Bentuk *endpoint* berubah (bukan array JSON, path berubah)	dianggap kegagalan sumber (*circuit breaker*)	Tidak

Tabel 2.2: Jenis perubahan skema sumber dan perilaku Aggregator.

Alternatif	Pertimbangan
Struct tetap	Sederhana dan bertipe, tetapi *field* baru hilang begitu saja atau seluruh pesan gagal dibaca.
Schema registry	Validasi terpusat, tetapi sumber harus ikut mendaftarkan skema. PVMBG justru mengubah skema *tanpa memberi tahu siapa pun*, sehingga registry tidak akan pernah mutakhir tepat waktu.
Protobuf atau Avro (schema evolution)	Evolusi skema terformalisasi, tetapi sumber mengirim JSON biasa. Lapisan konversi tetap harus mengetahui skema baru.
Pemetaan dinamis lewat berkas konfigurasi	Tidak perlu *redeploy* kode, tetapi seseorang tetap harus mengubah konfigurasi saat *field* baru muncul. Itu masih intervensi manual.
***Tolerant reader* dengan additive-only (dipilih)**	Bekerja *sepihak* dari sisi BNPB: tanpa koordinasi, tanpa *redeploy*, perilakunya deterministik dan mudah dibuktikan.

Tabel 2.3: Alternatif mekanisme untuk evolusi skema.

Sumber bersifat otonom dan tidak berkoordinasi, sehingga mekanisme yang dipilih harus berjalan tanpa kerja sama mereka. Perubahan aditif adalah jenis perubahan yang paling umum dan paling aman untuk ditangani otomatis. Perubahan yang mengubah arti kontrak (ganti nama, ganti tipe, enum baru) sengaja ditolak sejak awal, karena menebaknya berisiko salah menafsirkan data kebencanaan. Di sisi penyimpanan, `attributes` adalah dokumen JSONB, sehingga kebijakan ini tidak berubah menjadi migrasi tabel (Problem 4).

2.3 Batasan yang disadari

- Tipe nilai di `attributes` tidak divalidasi. Ini disengaja karena atribut memang tanpa skema, sehingga perubahan tipe pada *field* atribut lolos apa adanya.
- *Field* yang tidak dikenal ikut diteruskan, termasuk yang mungkin sensitif. Hal ini aman karena seluruh `attributes` berklasifikasi Mentah dan tidak pernah sampai ke Media (ditegakkan oleh Client-Facing API, Problem 3).
- *Field* yang dihapus sumber tetap menempel pada *record* lama karena penggabungan aditif. Hanya *record* lama yang terpengaruh, *record* baru tidak memilikinya. Konsekuensi ini kami terima agar informasi tidak pernah terbuang.
- Penggantian nama *field* tidak dikenali sebagai penggantian nama. *Field* lama dianggap hilang (dikarantina jika wajib) dan *field* baru muncul sebagai atribut.
- Perubahan arti dan satuan tidak terdeteksi. Tidak ada pemantauan *drift* selain *log* dan counter karantina.
- Karantina hanya disimpan di memori (200 entri terakhir, counter dihitung sejak start). Fungsinya sebagai alat diagnosis, bukan penyimpanan resmi, dan belum ada fitur untuk memutar ulang *record* yang dikarantina setelah kode diperbaiki.
- *Record* yang ditolak diingat selama 15 menit agar *polling* ulang pada jendela *overlap* tidak mencatatnya berulang kali.
- Pencocokan enum tidak membedakan huruf besar dan kecil (`SIAGA` sama dengan `Siaga`), dan nilai lain ditolak.

2.4 Bukti penyelesaian

Bukti berasal dari `make demo-p1` (transkrip: `docs/evidence/p1/latest.log`). Demo memeriksa hasilnya sendiri dan berakhir dengan `P1 PASS`.

Cuplikan log: skema berubah saat berjalan (langkah 4 dan 5)

```
$ POST http://localhost:8082/admin/schema-version {confidence_level:true, extra_fields:{
    lava_dome_volume_m3:125000}}
{"schema_version":2,"confidence_level":true,"extra_fields":{"lava_dome_volume_m3":125000}}
...
{"hazard_id":"pvmbg-VR-58ae09-0029","severity":"AWAS","area_name":"Gunung Lewotobi Laki-laki",
 "attributes":{"ash_column_height_m":5345.9,"confidence_level":0.78,"eruption_count_24h":18,
              "lava_dome_volume_m3":125000}}
   PASS bmkg-mock, pvmbg-mock and aggregator were NOT restarted (StartedAt unchanged)
   PASS all 28 pre-change volcanic hazards are unchanged (same version, same attributes)
   side by side in the store: 5 hazards WITH confidence_level, 28 WITHOUT - both readable
```

Cuplikan log: perubahan yang tidak didukung dikarantina (langkah 7)

```
$ POST http://localhost:8082/admin/generate {override:{alert_level:"Darurat"}}   # nilai enum baru
```

Kriteria pada spesifikasi	Bukti
Skema awal menghasilkan HazardEvent sesuai aturan, untuk kedua sumber	Langkah 2: pembanding independen membaca data mentah dari *mock*, menghitung sendiri HazardEvent yang diharapkan, lalu mencocokkannya dengan isi Canonical Store. Hasilnya 26 *event* BMKG dan 27 laporan PVMBG, semuanya lulus, dan keempat cabang severity terwakili (AWAS 2, NORMAL 12, SIAGA 3, WASPADA 9).
Skema berubah saat berjalan, tanpa *restart*	Langkah 4 memanggil `POST /admin/schema-version`. Pada langkah 5, waktu `StartedAt` bmkg-*mock*, pvmbg-*mock*, dan aggregator tidak berubah.
Tetap valid setelah perubahan, tanpa crash atau intervensi kode	Langkah 5: `confidence_level` dan *field* baru `lava_dome_volume_m3` muncul di `attributes`, pembanding lulus untuk 32 laporan, dan kedua sumber tetap up.
Record sebelum dan sesudah tersimpan berdampingan	Langkah 6: 28 hazard vulkanik lama tidak berubah (versi dan atribut sama). Di *store* ada 5 hazard *dengan* dan 28 *tanpa* `confidence_level`, dan keduanya terbaca.
Kebijakan konsisten	Langkah 5 (*field* baru diteruskan) dan langkah 7 (perubahan yang tidak didukung masuk karantina tanpa memblokir batch), serta `TestUnknownFieldPolicyIsConsistent`.
Log ber-*correlation ID* dan latensi	Langkah 8: satu `cid` melintasi `outbound_call` (dengan `latency_ms`), `hazard_stored`, `poll_ok`, dan `outbox_published`.

Tabel 2.4: Kriteria Problem 1 dan buktinya.

```
   PASS the valid report issued right after the bad one was stored (the batch was not blocked)
   PASS report VR-58ae09-0035 with alert_level=Darurat was NOT stored
{"source":"PVMBG","record_id":"VR-58ae09-0035","field":"alert_level",
 "reason":"unknown enum value \"Darurat\"","correlation_id":"5dacbefeb284a2f8"}
```

Cuplikan log: satu alur, satu correlation ID (langkah 8)

```
correlation id: 5dacbefeb284a2f8
{"msg":"outbound_call","target":"pvmbg","latency_ms":2228.447,"status":200}
{"msg":"mapping_rejected"}
{"msg":"hazard_stored","hazard_id":"pvmbg-VR-58ae09-0036","version":1,"outcome":"created"}
{"msg":"poll_ok","fetched":3,"created":1,"quarantined":1}
{"msg":"event_published","latency_ms":23}
{"msg":"outbox_published","hazard_id":"pvmbg-VR-58ae09-0036","version":1,"event_id":"pvmbg-VR-58
    ae09-0036:1"}
```

Tes otomatis yang terkait:

- `mapping_test.go`: tabel severity pada batas 4,99, 5,0, 6,49, dan 6,5, warning yang terlambat, idempotensi, dan *round-trip* JSON.
- `poller_test.go`: `TestPVMBGSchemaChangeWithoutRestart`, karantina yang tidak menahan *watermark*, dan deduplikasi.
- Tes server *mock* PVMBG (skema awal dan skema baru) serta tes kontrak JSON Schema di ketiga *service*.

2.5 Jawaban

Inti jawaban

Mekanisme yang dipilih. *Tolerant reader* dengan kebijakan *additive-only*. JSON dibaca secara generik, *field* kanonik divalidasi eksplisit, dan sisanya diteruskan ke `attributes`. *Schema registry* dan Protobuf atau Avro tidak dipilih karena sumber tidak berkoordinasi (alasan di bagian 2.2).
Kebijakan *field* yang tidak dikenal. Diteruskan ke `attributes`, tidak diabaikan.
Perubahan yang didukung. Penambahan *field* bertipe apa pun, hilangnya *field* yang hanya berstatus atribut, dan `volcano_id` baru (*graceful degradation*).
Perubahan yang sengaja tidak didukung. Penghapusan atau penggantian nama *field* wajib, perubahan tipe *field* wajib, dan nilai enum baru. Semuanya masuk karantina dan batch tetap berlanjut. Perubahan arti atau satuan tidak terdeteksi.

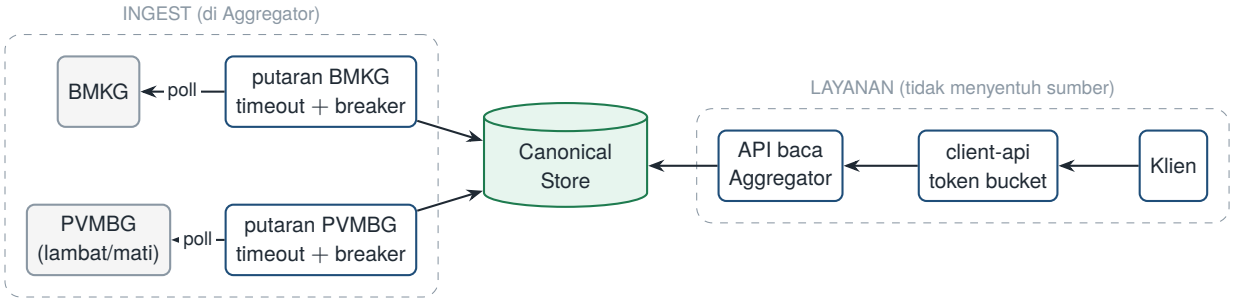
3 Problem 2 – Concurrency, Availability, dan Graceful Degradation

Ringkasan

Gateway tidak memanggil BMKG atau PVMBG ketika melayani klien. *Polling* berjalan di putaran terpisah untuk tiap sumber dan hasilnya disimpan di Canonical Store, sedangkan klien hanya membaca dari *store*. Dengan begitu, PVMBG yang lambat atau mati tidak bisa menahan permintaan lain. Lonjakan beban ditolak seketika dengan HTTP 429, dan saat PVMBG mati, data vulkanik terakhir tetap disajikan dengan penanda `stale`.

3.1 Rancangan solusi dan mekanisme

Idenya adalah memisahkan *pengambilan data* dari *pelayanan klien*. *Gateway* yang memanggil BMKG lalu PVMBG secara berurutan di dalam satu permintaan klien membuat lambatnya PVMBG dirasakan oleh semua permintaan. Pada rancangan kami, jalur layanan tidak pernah menyentuh sumber.



Gambar 3.1: Model konkurensi. Dua putaran polling menulis ke store; jalur layanan hanya membaca dari store.

1. **Satu putaran *polling* per sumber** (`poller/loop.go`). BMKG dan PVMBG masing-masing punya goroutine sendiri dengan *timeout* permintaan (`SOURCE_TIMEOUT_SEC`), *timeout* siklus, dan *circuit breaker* (`resilience/breaker.go`). Breaker terbuka setelah beberapa kegagalan beruntun, lalu mengirim satu percobaan (*half-open*) setelah `BREAKER_OPEN_SEC`. Siklus berikutnya baru dimulai `POLL_INTERVAL_SEC` setelah siklus sebelumnya selesai, sehingga sumber yang lambat tidak pernah menerima permintaan yang menumpuk.
2. **Jalur layanan tidak memanggil BMKG maupun PVMBG.** Client-Facing API membaca `GET /internal/*` milik Aggregator, yang membaca Canonical Store. Tiap permintaan klien dilayani goroutine `net/http` sendiri, sehingga permintaan data BMKG tidak berbagi antrean atau lock dengan apa pun yang menunggu PVMBG.
3. ***Rate limiter* dengan *token bucket*** di Client-Facing API (`CLIENT_RATE_LIMIT_RPS` dan `CLIENT_RATE_LIMIT_BURST`, nilai bawaan 500 dan 1000). Kelebihan beban *langsung ditolak* dengan status 429 dan *header* `Retry-After`, tidak dimasukkan ke antrean. *Endpoint* `/health` dan `/ready` dikecualikan agar pemeriksaan kesehatan tetap terjawab saat beban tinggi.
4. ***Graceful degradation.*** Aggregator menyimpan `freshness` untuk tiap sumber (`state`,

`last_success_at`, `stale_since`). Saat PVMBG mati, data vulkanik terakhir tetap disajikan dari *store*. Client-Facing API menandai tiap item dengan `stale: true` dan meneruskan `freshness.PVMBG.stale_since`. Tidak ada jalur yang mengubah matinya satu sumber menjadi error 500 atau permintaan yang menggantung.

3.1.1 Pencegahan tiga kegagalan

Kegagalan pada spesifikasi	**Mekanisme yang mencegahnya**
Head-of-line blocking	Tidak ada rantai panggilan berurutan BMKG lalu PVMBG. Putaran *polling* terpisah untuk tiap sumber, dan permintaan klien hanya menyentuh *store*. PVMBG yang butuh 3 detik tidak berada di jalur permintaan BMKG-only mana pun.
Resource exhaustion	*Token bucket* menolak kelebihan beban *sebelum* menyentuh Aggregator. Setiap panggilan keluar punya *timeout*. Breaker mencegah sumber yang mati menghabiskan goroutine dan koneksi, karena *polling* dilewati dan tidak ditumpuk. Batasnya dipasang pada *laju* permintaan, bukan pada jumlah permintaan yang sedang berjalan. Karena permintaan dilayani dari *store* lokal dalam hitungan milidetik (p99 sekitar 28 ms pada *load test*), pembatasan laju ikut membatasi jumlah permintaan yang berjalan bersamaan.
Tidak ada degradasi terkendali	Data terakhir dari Canonical Store disajikan bersama `stale` dan `stale_since`, sedangkan sumber lain tetap segar.

Tabel 3.1: Pemetaan tiga kegagalan ke mekanisme pencegahnya.

3.2 Alasan keputusan dan trade-off

Model konkurensi di atas bukan satu-satunya pilihan. Tabel berikut memuat alternatif yang kami pertimbangkan.

Alternatif	**Alasan tidak dipilih**
Memanggil BMKG dan PVMBG *secara paralel* pada tiap permintaan klien	Menghilangkan pemblokiran berurutan, tetapi permintaan tetap menunggu PVMBG yang paling lambat (3 detik) atau harus *timeout*. Setiap lonjakan klien juga melipatgandakan beban ke sumber yang sudah lemah.
Cache dengan TTL di Client-Facing API	Hanya menyembunyikan masalah. Data bisa basi tanpa penanda, dan satu fakta tersimpan di dua tempat (melanggar aturan satu *storage* satu pemilik).
Antrean dan *worker pool* untuk permintaan klien	Membatasi konkurensi, tetapi menukar penolakan cepat dengan latensi yang membengkak karena antrean memenuhi memori.
Semaphore konkurensi (batas permintaan yang sedang berjalan)	Efektif melindungi dari permintaan yang lambat. Pada sistem ini permintaan klien cepat (*store* lokal), sehingga laju adalah ukuran yang lebih tepat.
Putaran per sumber, *store* sebagai penyangga, *token bucket* (dipilih)	Terpisah secara struktural, dengan perilaku saat gagal yang deterministik dan dapat didemokan.

Tabel 3.2: Alternatif model konkurensi.

Konsekuensi dari pilihan ini ada dua. Pertama, data yang dilayani *selalu* berasal dari *store*, sehingga tingkat kesegarannya setara dengan interval *polling* (3 detik). Kedua, kebijakan laju bersifat global (dijelaskan pada bagian berikut).

3.3 Batasan yang disadari

- *Rate limiter* bersifat global dan tidak per klien, serta dipasang *sebelum* autentikasi. Satu klien yang agresif, atau banjir permintaan tanpa *token*, dapat menghabiskan jatah klien lain. Untuk M1 ini cukup membuktikan perlindungan sumber daya, dan pembatasan per klien menjadi perbaikan untuk M2.
- Tidak ada batas jumlah koneksi bersamaan selain laju. Beban koneksi lambat (*slowloris*) hanya dibatasi oleh `ReadHeaderTimeout` selama 5 detik.
- Client-Facing API dan Aggregator berjalan dengan satu *instance*. Skala horizontal, termasuk *rate limiter* terdistribusi, masuk ke M2.
- Penanda `stale` pada permintaan satu hazard membutuhkan satu panggilan tambahan ke `/internal/sources/status`.
- Data saat outage adalah potret terakhir, tanpa estimasi atau ekstrapolasi.
- Hasil *load test* berasal dari **satu mesin**: k6, Docker, dan semua *service* berbagi CPU pada satu mesin Windows 11. Angka absolutnya bukan klaim kapasitas produksi. *Throughput* pada uji sustained dibatasi oleh *think time* 0,1 detik per VU (sekitar 450 permintaan per detik untuk 50 VU), bukan oleh kemampuan server.

3.4 Konfigurasi dan tooling load test

Load test dijalankan dengan k6. Konfigurasi berikut dicatat agar hasilnya bisa diulang.

Item	Nilai
Tool	**k6** (biner lokal, atau `grafana/k6:0.54.0` lewat Docker jika biner tidak ada)
Skrip	`scripts/loadtest/p2.js`, dijalankan oleh `scripts/loadtest/run-p2.sh` (bagian dari `make demo-p2`, langkah E dan F)
Target	`GET /v1/hazards?hazard_type=SEISMIC&limit=20` pada Client-Facing API port 8080, dengan peran Lapangan
Uji *sustained*	**50 VU konstan selama 60 detik**, think time 0,1 detik (`K6_VUS`, `K6_DURATION`, `K6_THINK_S`)
Uji *overload*	150 VU selama 20 detik tanpa think time, sengaja melampaui batas laju 500 permintaan per detik
Autentikasi	Tiap VU *login* sendiri, lalu memperbarui *token* lewat `POST /v1/refresh` 10 detik sebelum `exp` (TTL 60 detik). Versi awal skrip memakai satu *token* untuk seluruh 60 detik sehingga kedaluwarsa *token* terhitung sebagai error. Hal ini sudah diperbaiki
Klasifikasi respons	**200** berarti sukses, **429** berarti penolakan terkendali (dilaporkan, tidak dihitung sebagai error), dan **tak tertangani** berarti selain keduanya (5xx, 401/403, *timeout*)
Ambang lulus	`unhandled < 1%` dan `p95 < 300 ms`. `make demo-p2` gagal jika ambang terlampaui
Kondisi sistem	PVMBG diberi *delay* 3 detik pada langkah A, dan seluruh stack berasal dari `make reset`

Tabel 3.3: Konfigurasi *load test*.

3.5 Bukti penyelesaian

Angka berasal dari `make demo-p2` dan `make demo-p2-aggregator`, dengan berkas mentah di `docs/evidence/p2/`. Angkanya berbeda dari satu run ke run lain, sedangkan yang ditegakkan

skrip adalah kriteria lulus atau gagalnya. Sebagai contoh, pada dua run berbeda p95 BMKG-only bernilai 62,9 ms dan 152,4 ms, dan keduanya jauh di bawah batas 300 ms. Angka pada tabel berasal dari run terbaru (8 Oktober 2026, setelah `make reset`).

Kriteria pada spesifikasi	Hasil	Berkas bukti
p95 BMKG-only di bawah 300 ms saat PVMBG *delay* 3 detik, bersamaan dengan permintaan data PVMBG	4 pembaca data vulkanik paralel + 40 permintaan BMKG-only: **p95 152,4 ms**, 0 error. Pembaca vulkanik p95 154–190 ms.	`latest.log`, langkah A
Load test ≥50 koneksi, ≥60 detik	26.752 permintaan, **445,1 req/s**, **p50 9,4 ms, p95 19,6 ms, p99 28,3 ms**, maksimum 69 ms. Status 200: 26.752, 429: 0, lainnya: 0. **Error tak tertangani 0%** (batas 1%). 50 perpanjangan *token* berjalan di tengah uji.	`k6-summary.json`
Overload terkendali (150 VU)	81.112 permintaan (sekitar 4.050 req/s): 200: 10.989, **429: 70.123 (86,5%)**, **lainnya: 0**, p95 68 ms. `/health` tetap 200 setelahnya dan tidak ada crash.	`k6-overload-summary.json`, `latest.log`, langkah F
Outage PVMBG (mode `error`)	27 item vulkanik tetap disajikan, **semuanya `stale: true`**, dan `freshness.PVMBG.stale_since` terisi. Pembacaan vulkanik p95 19,7 ms dengan 0 error, dan *event* BMKG baru tetap masuk.	`latest.log`, langkah B; `aggregator-latest.log`
Outage mode `hang` (tidak menjawab)	Pembacaan seismik p95 12,1 ms dan klien tidak menggantung. Di sisi Aggregator, p95 9,0 ms.	`latest.log`, langkah C
Pulih tanpa *restart*	PVMBG kembali up, laporan baru ter-ingest, dan `StartedAt` lima *container* tidak berubah.	`latest.log`, langkah D
Correlation ID lintas *service*	Satu `cid` melintasi `request` client-api, `outbound_call`, dan `request` aggregator.	`cid-trace.log`

Tabel 3.4: Kriteria Problem 2 dan buktinya.

Catatan tentang overload: dalam 20 detik, *token bucket* dengan 500 *token* per detik dan *burst* 1000 secara teori meloloskan sekitar $500 \times 20 + 1000 \approx 11.000$ permintaan. Hasil ukur 10.989 permintaan sukses sangat dekat dengan angka itu, yang berarti pembatas bekerja sesuai konfigurasi. Semua sisanya ditolak dengan 429 tanpa satu pun 5xx.

Sisi Aggregator saat PVMBG mati (`make demo-p2-aggregator`)

```
A. PVMBG answers in 3 s:   Aggregator API seismic p95 = 19.3 ms, non-200 = 0
   PASS a new BMKG event was ingested 2 s after it appeared, although PVMBG is slow
B. PVMBG outage (error):   PASS stale_since is set: 2026-10-07T19:44:08.985391266Z
   PASS all 87 volcanic hazards are still served from the Canonical Store (now 88)
   PASS volcanic reads during the outage: p95 77.5 ms, 0 errors (no 500, no hanging request)
   PASS while the circuit is open polls are skipped, not sent to the dead service (
     skipped_by_breaker=44)
C. PVMBG outage (hang):    PASS Aggregator API answers in p95 9.0 ms
   last_error: ... net/http: request canceled (Client.Timeout exceeded while awaiting headers)
D. Outage ends:            PASS PVMBG is 'up' again, stale_since=null
   PASS nothing was restarted (StartedAt of bmkg-mock, pvmbg-mock, aggregator unchanged)
```

Satu correlation ID melintasi dua service (`cid-trace.log`)

```
{"msg":"request","service":"aggregator","path":"/internal/hazards","status":200,"duration_ms
    ":0.885,"cid":"demo-p2-trace-0001"}
```

```
{"msg":"outbound_call","service":"client-api","target":"aggregator","latency_ms":1.463,"status
    ":200,"cid":"demo-p2-trace-0001"}
{"msg":"request","service":"client-api","path":"/v1/hazards","status":200,"duration_ms":2.107,"cid
    ":"demo-p2-trace-0001"}
```


Tes otomatis yang terkait:

- `TestSlowSourceDoesNotBlockFastSourceAndCyclesNeverOverlap`: sumber yang lambat tidak menahan sumber yang cepat.
- `TestStaleMarkerWhenSourceIsDown`: penanda `stale` muncul untuk semua peran.
- `TestUpstreamDownIs503NotHang`: jika Aggregator tidak terjangkau, permintaan dijawab cepat dan tidak menggantung.
- `TestShedsExcessWith429ButKeepsHealthAlive`: kelebihan beban ditolak dengan 429 dan `/health` tetap hidup.
- `resilience_test.go`: perilaku *circuit breaker*.

3.6 Jawaban

Inti jawaban

Model konkurensi. Putaran *polling* terpisah untuk tiap sumber (dengan *timeout* dan *circuit breaker*) menulis ke Canonical Store. Jalur layanan hanya membaca *store* lewat API Aggregator, dengan *token bucket* di depannya. Pemetaan ke tiga kegagalan ada di bagian 3.1.1.
Konfigurasi dan tooling *load test*. Memakai k6 dengan 50 VU selama 60 detik (sustained) dan 150 VU selama 20 detik (overload). Respons diklasifikasi menjadi 200, 429, atau tak tertangani, dengan ambang `unhandled < 1%` dan `p95 < 300 ms`. Rinciannya ada di bagian 3.4.

4 Problem 3 – Autentikasi dan Kepercayaan antar-Service

Ringkasan

Setiap domain kepercayaan memiliki kredensial sendiri yang diterbitkan dan diperiksa oleh pemiliknya, dan tidak ada satu pun rahasia yang dipakai di dua domain. Kunci BMKG tidak membuka PVMBG, kunci PVMBG tidak membuka BMKG, dan *token* klien tidak membuka keduanya. Untuk klien, Auth Service menerbitkan JWT berumur 60 detik untuk tiap peran, dan Client-Facing API menegakkan hak akses *di sisi server*: Media tidak pernah menerima *field* Mentah.

4.1 Rancangan solusi dan mekanisme

Ada lima domain kepercayaan, dan masing-masing dibuktikan terpisah pada bagian bukti.

Domain	Kredensial	Diperiksa oleh	Jika bocor
BMKG	`X-BMKG-Key`, kunci opaque	bmkg-*mock*	hanya BMKG
PVMBG	`Authorization: Bearer pvmbg.v1.<klaim>.<HMAC>`, dengan struktur berbeda	pvmbg-*mock*	hanya PVMBG
API internal Aggregator	`X-Service-Key`	Aggregator	jalur internal saja; tidak membuka BMKG, PVMBG, atau klien
Operator *mock*	`X-Admin-Token`, satu per *mock*	masing-masing *mock*	`/admin/*` pada *mock* itu saja
Klien downstream	`client_id` dan `client_secret` per peran, lalu JWT	Auth Service (penerbit) dan Client-Facing API (pemeriksa)	hanya peran klien itu

Tabel 4.1: Lima domain kepercayaan dan kredensialnya masing-masing.

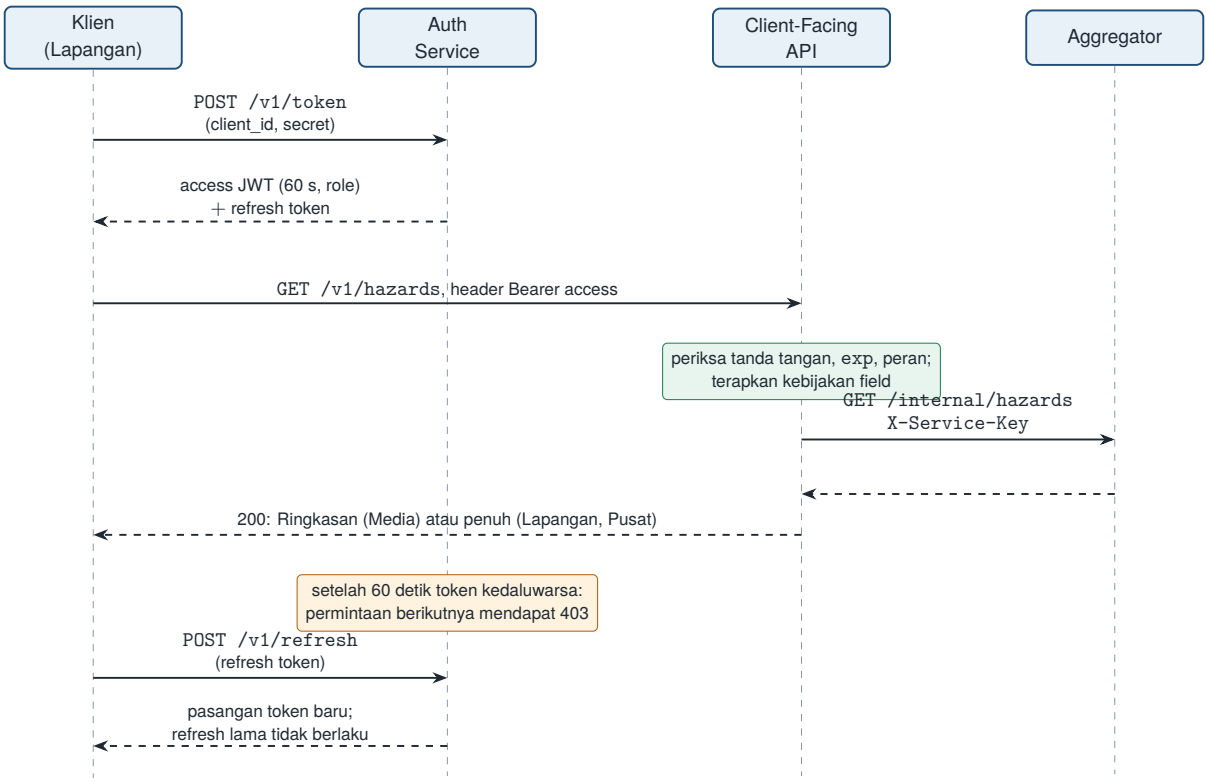
Aggregator memegang *token* PVMBG, tetapi **tidak memegang rahasia penanda tangannya**. Akibatnya Aggregator bisa memanggil PVMBG, tetapi tidak bisa mencetak *token* baru atau mengadministrasi PVMBG.

4.1.1 Klien downstream: Media, Tim Lapangan, BNPB Pusat

- **Tiga identitas yang benar-benar berbeda** (`AUTH_MEDIA_*`, `AUTH_LAPANGAN_*`, `AUTH_PUSAT_*`, masing-masing dengan id dan secret sendiri), bukan satu akun dengan *flag* peran di sisi klien. Peran ditentukan Auth Service dari identitas yang berhasil *login*, lalu ditulis ke klaim `role` *di dalam token yang ditandatangani*. Klien tidak dapat memilih perannya sendiri.
- **Access *token*** berupa JWT HS256 dengan klaim `sub`, `role`, `jti`, `iat`, dan `exp`. TTL-nya **60 detik** (`AUTH_ACCESS_TTL_SEC`, dapat diubah).
- ***Refresh token*** berupa 256 bit acak yang **tidak pernah disimpan dalam bentuk asli**. Yang

disimpan hanya hash SHA-256 di SQLite milik Auth Service (volume yang hanya dipasang pada *container* itu). *Token* ini sekali pakai: `POST /v1/refresh` menandai *token* lama sebagai `revoked` dalam transaksi yang sama dengan pembacaannya, lalu menerbitkan pasangan *token* baru. TTL-nya `AUTH_REFRESH_TTL_SEC` (24 jam).

- **Penegakan di sisi BNPB pada setiap permintaan** (`services/client-api`):
  1. Tanpa `Authorization: Bearer` berarti **401**. *Token* rusak, kedaluwarsa, salah tanda tangan, atau berperan tidak dikenal berarti **403**.
  2. **Media** hanya menerima *field* Ringkasan lewat *whitelist* (`policy.FilterItem`). *Field* baru yang kelak muncul di HazardEvent tidak otomatis bocor ke Media.
  3. Permintaan Media atas *field* Mentah (`fields=latitude`, `include=mentah`, dan sejenisnya) **ditolak dengan 403**. Aturannya juga berbasis *whitelist* (`policy.CheckQuery`): setiap nilai `fields` harus berupa *field* Ringkasan, dan `include` hanya boleh kosong atau `summary`. Variasi penulisan (huruf besar, spasi, `fields[]`, parameter berulang, daftar dipisah koma, nama yang tidak dikenal) tidak lolos.
  4. `/v1/sources/status` hanya untuk Lapangan dan Pusat, sedangkan Media mendapat 403.
- Kredensial, kunci penandatangan, dan konfigurasi dibaca dari *environment variable*. *Service* berhenti dengan pesan yang jelas jika variabelnya kosong (`obs.MustEnv`). Repositori hanya memuat `.env.example` yang berisi nilai contoh.



Gambar 4.1: Alur autentikasi klien: login, akses, kedaluwarsa, lalu *refresh* tanpa login ulang.

4.2 Alasan keputusan dan trade-off

Tabel berikut membandingkan beberapa cara mengamankan akses klien dengan pilihan kami.

Pilihan ini punya konsekuensi. `AUTH_JWT_SECRET` dipegang oleh *dua service* milik BNPB, yaitu

Alternatif	Alasan tidak dipilih
Satu kunci untuk semua	Jika kunci Media bocor, data mentah ikut bocor, dan kunci BMKG bisa menembus PVMBG. Ini justru masalah yang harus diselesaikan.
API key statis per klien, tanpa *token*	Tidak ada kedaluwarsa dan *refresh*, padahal Tim Lapangan wajib memakai *token* berumur pendek. Kunci yang bocor akan berlaku selamanya.
Token opaque dengan introspeksi ke Auth Service pada tiap permintaan	Pencabutan bisa seketika, tetapi setiap permintaan klien menambah satu panggilan jaringan dan Auth Service menjadi titik gagal bagi jalur baca. Hal ini bertentangan dengan tujuan Problem 2.
JWT asimetris (RS256 atau EdDSA)	Lebih aman, karena Client-Facing API cukup memegang kunci publik sehingga penyerang yang berhasil membobol API tidak bisa mencetak *token*. Namun pengelolaan kuncinya lebih rumit. **Dicatat sebagai perbaikan untuk M2.**
OAuth2 atau Keycloak lengkap	Berlebihan untuk tiga klien *read-only*, dan sulit didemokan dengan TTL 60 detik.
JWT HS256 dengan *refresh token* opaque (dipilih)	Verifikasi dilakukan lokal (jalur baca tidak bergantung pada Auth Service), TTL pendek, dan *refresh token* dapat dicabut per *token*. Sederhana dan mudah dibuktikan.

Tabel 4.2: Alternatif mekanisme autentikasi klien.

Auth Service dan Client-Facing API. Rahasia ini independen dari kredensial BMKG, PVMBG, kunci layanan, dan *token* admin, sehingga kebocorannya tidak merambat ke domain lain. Namun jika Client-Facing API berhasil dibobol, penyerang dapat mencetak *token* klien. Karena itu JWT asimetris dicatat sebagai langkah berikutnya.

4.3 Batasan yang disadari

- Access *token* lama tidak dicabut seketika saat *refresh*. JWT bersifat *stateless*, sehingga *token* yang belum kedaluwarsa tetap berlaku sampai `exp` (paling lama 60 detik). Yang dicabut seketika adalah *refresh token* lama (terbukti: jika dipakai ulang, hasilnya 403). Pencabutan seketika menuntut introspeksi atau daftar `jti` yang selalu diperbarui.
- Tidak ada deteksi pemakaian ulang *refresh token*. Satu keluarga *token* tidak dicabut ketika *refresh token* yang sudah terpakai muncul lagi.
- `/v1/token` tidak membatasi jumlah percobaan (tanpa lockout atau *rate limit* per klien). Perbandingan secret memakai waktu konstan.
- *Rate limiter* Client-Facing API bersifat global dan dipasang sebelum autentikasi, sehingga banjir permintaan tanpa *token* ikut menghabiskan jatah. Pembatasan per klien setelah autentikasi menjadi perbaikan untuk M2.
- *Token* kedaluwarsa dijawab dengan **403**, bukan 401, mengikuti kontrak yang dibekukan di `docs/contracts.md`.
- Tidak ada TLS di dalam jaringan Compose. Secret klien bersifat statis di *environment* tanpa perkakas rotasi. SQLite pada Auth Service hanya cocok untuk satu *instance*, sehingga M2 membutuhkan penyimpanan bersama.

4.4 Bukti penyelesaian

Setiap kriteria pada spesifikasi dibuktikan oleh demo yang memeriksa hasilnya sendiri atau oleh tes otomatis.

Kriteria pada spesifikasi	Bukti
Kredensial BMKG ditolak PVMBG dan sebaliknya (401/403)	`make demo-mock-auth`: kunci BMKG sebagai Bearer → 403, *token* PVMBG di `X-BMKG-Key` → 403, *token* yang dirusak → 403, tanpa kredensial → 401, dan *token* admin silang → 403. Setiap *container* hanya memegang rahasianya sendiri.
Media hanya menerima Ringkasan, dan permintaan *field* mentah ditolak server	`make demo-p3` langkah B (isi respons tanpa `latitude`, `longitude`, `attributes`, dan `source_ref_id`) dan langkah C (sepuluh bentuk permintaan Mentah → 403, permintaan yang hanya meminta Ringkasan → 200). Tes: `TestMediaNeverReceivesMentahOnListOrGet` dan `TestMediaExplicitMentahRequestIs403`.
Token Tim Lapangan kedaluwarsa secara alami, *refresh* tanpa *login* ulang, *token* lama ditolak	`make demo-p3` langkah F: *token* valid → tunggu sampai `exp` → 403 → `POST /v1/refresh` → 200 dengan *token* baru *tanpa* `client_secret` → *refresh token* lama dipakai ulang → 403.
Tidak ada kredensial yang ter-hardcode atau ter-*commit*, dan `.env.example` tersedia	`make secrets`: `.env` tidak pernah ter-*commit*, gitleaks bersih (riwayat *commit* dan *working tree*), tidak ada literal kredensial di kode Go non-test, dan `.env.example` memuat seluruh variabel.
401 dibedakan dari 403	`make demo-p3` langkah A dan `TestMissingTokenIs401AndBadTokenIs403`: tanpa *token* menghasilkan 401, sedangkan *token* sampah, kedaluwarsa, kunci lain, tanpa `exp`, peran tidak dikenal, dan `alg=none` semuanya menghasilkan 403.

Tabel 4.3: Kriteria Problem 3 dan buktinya.

Kredensial saling menolak (`make demo-mock-auth`)

```
[1] BMKG mock (X-BMKG-Key)
   PASS own key                          -> HTTP 200
   PASS no credential                    -> HTTP 401
   PASS wrong key                        -> HTTP 403
   PASS PVMBG bearer token               -> HTTP 403
[2] PVMBG mock (Authorization: Bearer <pvmbg-token>)
   PASS own token                        -> HTTP 200
   PASS BMKG key presented as bearer     -> HTTP 403
   PASS tampered PVMBG token             -> HTTP 403
   claims: {"aud":"pvmbg-api","exp":...,"iss":"pvmbg-geologi","sub":"bnpb-aggregator"}
[5] Each container only holds the secrets of its own trust domain
   PASS bmkg-mock has no PVMBG/internal secret; pvmbg-mock has no BMKG/internal secret;
        the aggregator holds tokens but cannot mint or administer
```

Media, Mentah, dan token kedaluwarsa (`make demo-p3`)

```
[1] PASS no header -> 401        PASS empty Bearer -> 401        PASS garbage JWT -> 403
[2] PASS media list omits all Mentah fields on first item
[3] PASS media ?fields=latitude -> 403         PASS media ?include=mentah -> 403
    PASS media ?fields=Latitude -> 403         PASS media ?fields%5B%5D=longitude -> 403
    PASS media ?include=raw -> 403             PASS media asking only for Ringkasan fields -> 200
[4] PASS media /v1/sources/status -> 403       PASS lapangan /v1/sources/status -> 200
[6] expires_in=60s (configured AUTH_ACCESS_TTL_SEC=60)
    PASS fresh access token works
    PASS expired access token -> 403
    PASS refreshed access works without client_id/secret again
    PASS rotated refresh token cannot be reused (403)
```

4.5 Jawaban

Mekanisme yang dipakai

Mekanisme terdiri dari dua lapis. **(1) Antarinstansi:** kredensial BMKG (kunci opaque) dan PVMBG (*token* yang ditandatangani dengan HMAC dan berstruktur lain, dengan rahasia penandatangan hanya ada di *mock* PVMBG) tidak saling berlaku, dan masing-masing diperiksa oleh pemiliknya. **(2) Klien downstream:** Auth Service menerbitkan JWT HS256 berumur 60 detik untuk tiap peran, ditambah *refresh token* opaque yang sekali pakai. Client-Facing API memeriksa setiap permintaan dan menegakkan klasifikasi Ringkasan dan Mentah di sisi server.

Mengapa tidak membuka celah baru

- **Pemalsuan *token*.** Algoritma dikunci ke HS256 (`alg=none` dan algoritma lain ditolak, dan sudah diuji). Klaim `exp` wajib ada dan diperiksa (*token* bertanda tangan tanpa `exp` ditolak, dan sudah diuji). Peran harus salah satu dari tiga nilai yang dikenal.
- **Eskalasi peran dari sisi klien.** Peran hanya berasal dari klaim yang ditandatangani. *Header* atau parameter buatan klien (misalnya `X-Role`) tidak dibaca. Media tidak punya jalur ke *field* Mentah karena respons dibentuk dengan *whitelist*, bukan dengan membuang *field* yang dikenal.
- **Kebocoran lintas domain.** Rahasia tiap domain terpisah dan tidak dipakai ulang. Aggregator tidak memegang rahasia penandatangan PVMBG (diverifikasi oleh `demo-mock-auth`).
- **Pencurian dari penyimpanan.** *Refresh token* hanya tersimpan sebagai hash, dan basenya tidak dapat dijangkau *service* lain. Dampak pencurian *refresh token* dibatasi oleh sifatnya yang sekali pakai dan oleh TTL.
- **Jendela penyalahgunaan.** *Token* yang dicuri hanya berguna paling lama 60 detik (access *token*) atau satu kali pakai (*refresh token*).
- **Batasan yang diakui** ada di bagian 4.3: tidak ada pencabutan access *token* seketika, dan rahasia HS256 dipegang dua *service* BNPB.

5 Problem 4 – Arsitektur Microservice dan Flexible Storage

Ringkasan

Sistem dipecah menjadi *container* yang masing-masing hanya punya satu alasan untuk berubah: *mock* sumber, Aggregator, Auth Service, Client-Facing API, dan *consumer*. Canonical Store memakai **PostgreSQL dengan kolom JSONB**, sehingga *field* baru seperti `confidence_level` tidak membutuhkan perubahan tabel. Jaringan Docker *internal* membuat database **secara fisik tidak terjangkau** oleh *service* selain Aggregator.

Status bukti Problem 4

Ketiga kriteria dibuktikan oleh satu skrip, `make demo-p4`. Pada langkah A, `pvmbg-mock` dihentikan, dibangun ulang, dan dijalankan lagi sementara *container* lain tetap berjalan. Pada langkah B sampai D, *field* baru masuk ke PostgreSQL JSONB tanpa migrasi dan data lama tetap terbaca. Pada langkah E, terbukti hanya Aggregator yang dapat menjangkau Canonical Store.

5.1 Rancangan solusi dan mekanisme

5.1.1 Pembagian service

Batas antarservice ditarik dengan satu aturan: **satu *container* punya satu alasan untuk berubah, dan setiap data punya satu pemilik**. Perbaikan untuk satu alasan tidak boleh memaksa *container* lain di-*rebuild*.

Container	Tanggung jawab tunggal	Berubah ketika
`bmkg-mock`, `pvmbg-mock`	Menyimulasikan lembaga lain dan bukan bagian BNPB	kontrak sumber berubah (di luar kendali BNPB)
`aggregator`	Mengambil data, memetakannya, memiliki Canonical Store, dan mempublikasikan *event*	aturan pemetaan atau ingest berubah
`auth-service`	Menerbitkan dan memperbarui *token* klien	kebijakan autentikasi berubah
`client-api`	Menyajikan data serta menegakkan hak akses dan *rate limit*, tanpa *state*	kebijakan *field* atau *rate limit* berubah
`consumer-*`	Melayani satu kebutuhan pihak yang berlangganan (dashboard, notifikasi)	kebutuhan pihak tersebut berubah
`postgres-canonical`, `kafka`	Infrastruktur penyimpanan dan distribusi *event*	versi atau konfigurasi infrastruktur berubah

Tabel 5.1: Batas *service* dan alasan masing-masing berubah.

Spesifikasi memberi contoh *perbaikan kecil pada autentikasi*. Perbaikan seperti itu hanya menyentuh `auth-service` (atau `client-api` untuk bagian pemeriksaannya). Aggregator, *mock*, dan *consumer* tidak ikut di-*rebuild*.

5.1.2 Aturan kepemilikan penyimpanan

Setiap penyimpanan hanya dimiliki oleh satu *service*. Komponen lain meminta data lewat API milik pemiliknya.

Penyimpanan	Pemilik	Cara komponen lain memperoleh data
Canonical Store (PostgreSQL)	`aggregator`	API internal Aggregator (`/internal/*`) dengan `X-Service-Key`
Hash *refresh token* (SQLite)	`auth-service`	*endpoint* `/v1/token` dan `/v1/refresh`
Topic Kafka `hazard.events`	`aggregator` sebagai publisher	men-*subscribe* sebagai *consumer group*
Kumpulan `event_id` di *consumer*	masing-masing *consumer*	tidak dibagikan

Tabel 5.2: Kepemilikan penyimpanan.

5.1.3 Penyimpanan yang fleksibel: PostgreSQL dengan JSONB

Tabel `hazards` hanya memakai kolom tetap untuk identitas dan *field* kanonik. Semua yang bisa berubah masuk ke satu kolom `attributes` bertipe JSONB.

```
CREATE TABLE IF NOT EXISTS hazards (
  hazard_id     text PRIMARY KEY,
  source        text NOT NULL,     source_ref_id text NOT NULL,
  hazard_type   text NOT NULL,     severity      text NOT NULL,
  area_name     text NOT NULL,
  latitude      double precision,  longitude     double precision,
  occurred_at   timestamptz NOT NULL,  ingested_at timestamptz NOT NULL,
  updated_at    timestamptz NOT NULL,  version     integer NOT NULL,
  attributes    jsonb NOT NULL DEFAULT '{}'      -- field dinamis: tanpa DDL saat field baru muncul
);
```

Potongan di atas sudah disederhanakan. Definisi aslinya (`adapters/postgres/repo.go`) juga memuat indeks dan aturan pengurutan teks untuk `hazard_id`, tetapi tidak ada kolom lain. Perintah `CREATE ... IF NOT EXISTS` dijalankan *sekali saat container start*, bukan sebagai langkah migrasi setiap kali sumber berubah. Repositori tidak memuat direktori migrasi maupun berkas SQL terpisah. Ketika PVMBG mulai mengirim `confidence_level`, Aggregator tidak di-*restart* dan tidak ada DDL yang dijalankan, karena *field* itu langsung masuk ke `attributes`.

5.1.4 Isolasi jaringan sebagai bukti kepemilikan

Klaim bahwa hanya Aggregator yang memegang database tidak dibiarkan sebagai janji. Compose membuat empat jaringan, dan tiga di antaranya bersifat *internal* (tidak bisa diakses dari luar dan tidak bisa dipakai untuk membuka port ke host).

5.2 Alasan keputusan dan trade-off

Jaringan	Internal?	Anggota
`app-net`	tidak	*mock*, aggregator, auth-*service*, client-api, *consumer* (hanya port `/health` dan `/stats`)
`canonical-net`	ya	**hanya** aggregator dan postgres-canonical
`broker-net`	ya	kafka, aggregator (publisher), *consumer*
`auth-net`	ya	auth-*service*

Tabel 5.3: Jaringan Docker dan anggotanya.

5.2.1 Batas service: mengapa dipecah seperti ini

Sistem bisa dipotong menjadi *service* dengan beberapa cara. Tabel berikut membandingkannya dengan pilihan kami.

Pembagian alternatif	Pertimbangan
Monolit modular (satu *container*)	Paling sederhana, tetapi inilah kondisi yang menjadi masalah pada spesifikasi: perbaikan autentikasi memaksa seluruh sistem di-*rebuild*.
Dipecah per lapisan teknis (poller, mapper, penyimpan, publisher)	Terlalu halus. Hazard dan baris *outbox* harus ditulis dalam *satu transaksi*, dan jika dipisah, transaksi itu menjadi transaksi terdistribusi. Pemetaan juga membutuhkan hasil *polling* yang sama, sehingga memisahkannya hanya menambah satu lompatan jaringan tanpa manfaat *deploy* mandiri.
Dipecah per sumber (adapter BMKG dan adapter PVMBG masing-masing di *container* sendiri)	Menarik, karena perubahan skema PVMBG hanya me-*rebuild* adapter PVMBG. Tidak diambil pada M1 sebab setiap adapter butuh kontrak, *container*, dan jalur ke *store* tersendiri, sedangkan di dalam Aggregator kedua sumber sudah berjalan di putaran terpisah. **Kandidat untuk M2.**
Dipecah per alasan berubah (dipilih)	Sumber, Aggregator (pemilik data), autentikasi, penyajian, dan *consumer* dapat di-*rebuild* secara mandiri. Pasangan yang harus konsisten secara transaksi tetap berada di satu *container*.

Tabel 5.4: Alternatif pembagian *service*.

Konsekuensinya, mengubah aturan pemetaan berarti me-*rebuild* Aggregator, termasuk *polling*-nya. Hal ini tidak berbahaya karena penanda kemajuan *polling* (*watermark*) tersimpan di database dan Aggregator tidak bergantung pada urutan start (tanpa `depends_on` ke *mock* maupun *store*, dan koneksi dicoba ulang sampai 90 detik). Auth Service, Client-Facing API, dan *consumer* tidak ikut tersentuh.

5.2.2 Pilihan penyimpanan

Kebutuhan utamanya adalah *field* baru dari sumber tidak boleh memaksa migrasi tabel. Tabel berikut membandingkan pilihan yang masuk akal.

Konsekuensi dari pilihan ini:

- **Keuntungan:** *field* baru tidak membutuhkan migrasi, hazard dan entri *outbox* ditulis secara atomik (dasar bagi Problem 5), dan *keyset pagination* didukung oleh indeks `(hazard_type, occurred_at, hazard_id)`.
- **Kerugian:** isi `attributes` tidak divalidasi tipenya dan belum diindeks, sehingga API belum mendukung penyaringan berdasarkan nilai atribut. Jika kelak dibutuhkan, indeks GIN dapat ditambahkan tanpa mengubah kontrak.
- **Keputusan kecil yang disengaja:** kolom `outbox.payload` bertipe `json`, bukan `jsonb`, agar

Alternatif	Kelebihan	Konsekuensi bagi sistem ini
Relasional dengan kolom tetap	Skema tegas dan *query* kuat	Setiap *field* baru membutuhkan migrasi dan *deploy* bersama, sehingga masalah Problem 1 muncul kembali lewat jalur *storage*. **Ditolak.**
Tabel atribut terpisah (EAV)	Fleksibel tanpa DDL	*Query* menjadi rumit dan lambat (banyak *join*) dan tipe data hilang. Lebih buruk daripada JSONB.
Document *store* (misalnya MongoDB)	Cocok untuk dokumen tanpa skema	Menambah satu teknologi baru. Menulis hazard dan *outbox* secara atomik membutuhkan transaksi lintas dokumen (perlu *replica set*), yang lebih berat untuk satu *instance* di Compose.
Key-value (misalnya Redis)	Sangat cepat	Tidak mendukung *query* berurutan dan paginasi berbasis kunci yang kami butuhkan, serta tidak ada transaksi hazard dan *outbox*.
PostgreSQL dengan JSONB (dipilih)	Kolom tetap untuk data yang pasti dan JSONB untuk yang berubah, transaksi ACID, indeks dan *query* lengkap, serta satu teknologi untuk hazard dan *outbox*	Lihat konsekuensi di bawah.

Tabel 5.5: Perbandingan alternatif penyimpanan.

pesan tersimpan persis seperti saat dibuat (JSONB menata ulang urutan kunci).

5.3 Batasan yang disadari

- Demo *rebuild* memakai `pvmbg-mock`, bukan Aggregator. *Mock* itu dipilih karena paling aman dimatikan saat demo berjalan. Me-*rebuild* Aggregator berarti *polling* berhenti sebentar, dan hal itu tidak diperagakan oleh skrip ini.
- Selama PVMBG di-*rebuild*, data vulkanik tidak bertambah sampai *polling* pulih. Data seismik dan *record* lama tetap tersedia dari Canonical Store.
- Aggregator masih menggabungkan beberapa tugas (*polling*, pemetaan, kepemilikan *store*, *relay*, dan API internal). Ini keputusan yang disengaja (alasannya di atas), tetapi berarti *container* tersebut relatif besar.
- `pkg/obs` dipakai bersama oleh semua *service* (*log*, *correlation ID*, *health*). Isinya hanya utilitas teknis tanpa logika bisnis, sehingga sesuai aturan pada spesifikasi. Meski begitu, perubahan di sana tetap berarti semua *service* perlu di-*rebuild* agar memakainya.
- PostgreSQL dan Kafka masing-masing hanya satu *instance* tanpa replika. Ketahanan terhadap kegagalan bersamaan menjadi bahan untuk M2.
- Port API internal Aggregator dibuka ke *loopback* host (127.0.0.1:8083) untuk operator dan skrip demo, dan tidak dipakai oleh klien.
- Karantina disimpan di memori, bukan di Canonical Store.

5.4 Bukti penyelesaian

Tabel berikut memuat tiap kriteria pada spesifikasi beserta status buktinya saat ini.

Kriteria pada spesifikasi	Bukti	Status
Setiap komponen berjalan sebagai *container* terpisah	Setiap *service* punya `Dockerfile` sendiri dan satu perintah (`docker compose up --build`) menyalakan semuanya. Daftar *container* ada di Tabel 1.3.	**Lulus**
Satu *container* dapat dihentikan, di-*rebuild*, dan dijalankan ulang tanpa me-*restart container* lain, dan sistem tetap melayani permintaan lain	`make demo-p4`, langkah A: `pvmbg-mock` dihentikan, di-*rebuild* (`docker compose build`), lalu dijalankan lagi dengan `docker compose up -d --no-deps`. Selama proses itu, permintaan khusus data BMKG lewat Client-Facing API tetap berstatus 200. ID *container* dan waktu start delapan *container* lain dicatat sebelum dan sesudah, dan harus identik.	**Lulus**
HazardEvent tersimpan tanpa migrasi saat *field* baru muncul, dengan *record* lama dan baru berdampingan	`make demo-p4`, langkah B sampai D. Laporan lama (tanpa `confidence_level`) disimpan, skema PVMBG diubah saat sistem berjalan, lalu laporan baru disimpan. Keduanya ada di PostgreSQL dan terbaca lewat Aggregator, dan atribut laporan lama tidak berubah. *Fingerprint* kolom tabel `hazards` sama sebelum dan sesudah, begitu pula waktu start Aggregator, sehingga tidak ada DDL dan tidak ada *restart*. Demo Problem 1 menunjukkan hal serupa pada skala lebih besar (5 hazard dengan dan 28 tanpa *field* itu).	**Lulus**
Canonical Store hanya dapat diakses langsung oleh Aggregator	`make demo-isolation`, yang juga dijalankan sebagai langkah E pada `make demo-p4`. Cuplikannya ada di bawah.	**Lulus**

Tabel 5.6: Kriteria Problem 4 dan buktinya.

Isolasi Canonical Store (`make demo-isolation`)

```
bnpb_app-net:       aggregator auth-service bmkg-mock client-api consumer-dashboard consumer-
    notifier pvmbg-mock
bnpb_canonical-net: aggregator postgres-canonical
bnpb_broker-net:    aggregator consumer-dashboard consumer-notifier kafka

   PASS canonical-net contains exactly: aggregator + postgres-canonical
   PASS postgres publishes no port to the host
   PASS container on app-net (where mocks / client-facing API live): postgres-canonical:5432 is
    unreachable
   PASS container on broker-net (consumers): postgres-canonical:5432 is unreachable
   PASS control: a container on canonical-net (i.e. the Aggregator's network) does reach it
   PASS container on app-net: kafka:9092 is unreachable
   PASS the Aggregator API demands its service key (401 without, 403 with a wrong one)
ISOLATION PASS
```

Baris *control* penting karena membuktikan bahwa penolakan di atas *bukan* akibat pengujian yang rusak. *Container* percobaan di jaringan yang sama dengan Aggregator berhasil terhubung, sedangkan *container* di jaringan lain tidak.

Field baru tanpa migrasi (`make demo-p4`, langkah B–D)

```
[2] B. Store an old-schema volcanic report
    old row attributes: {"eruption_count_24h": 3, "ash_column_height_m": 2730}
    hazards table column fingerprint before: 4555805e1c5790a6fff3c9ce2923df04
    PASS pvmbg-VR-0c2b58-0025 stored without confidence_level in PostgreSQL JSONB
[3] C. Change PVMBG's schema live and store a new report without a migration
    new row attributes: {"confidence_level": 0.82, "eruption_count_24h": 15,
                         "ash_column_height_m": 1822.6}
    hazards table column fingerprint after: 4555805e1c5790a6fff3c9ce2923df04
    PASS pvmbg-VR-0c2b58-0028 contains confidence_level; table columns and
```

```
         Aggregator start time did not change
[4] D. Read old and new records side by side through the Aggregator
    PASS old=VR-0c2b58-0025 and new=VR-0c2b58-0028 coexist and are readable;
         old attributes are unchanged
```

Fingerprint yang sama sebelum dan sesudah berarti daftar kolom tabel tidak berubah. *Field* baru hanya muncul di dalam kolom `attributes`.

Me-rebuild satu service (`make demo-p4`, langkah A)

```
[1] A. Rebuild only PVMBG while unrelated services keep serving requests
    unrelated containers before rebuild:  (8 baris: nama | ID container | waktu start)
    unrelated containers after rebuild:   (8 baris, identik)
    PASS BMKG-only requests succeeded; IDs and start times of all other
         containers stayed fixed
```

5.5 Jawaban

Alasan penentuan batas *service*

Batas ditarik menurut *alasan berubah* dan *kepemilikan data*. Sumber, Aggregator (pemilik Canonical Store), autentikasi, penyajian, dan *consumer* dipisah karena masing-masing berubah karena alasan yang berbeda. *Polling*, pemetaan, penyimpanan, dan *outbox* tetap berada di satu *container* karena hazard dan baris *outbox* harus ditulis dalam satu transaksi. Pembagian per lapisan teknis ditolak karena mengubah transaksi itu menjadi transaksi terdistribusi, sedangkan pembagian per sumber menjadi kandidat untuk M2. Rinciannya ada di bagian 5.2.1.

Perbandingan alternatif penyimpanan

Alternatif yang dibandingkan adalah relasional dengan kolom tetap, EAV, document *store*, key-value, dan PostgreSQL dengan JSONB (bagian 5.2.2). PostgreSQL dengan JSONB dipilih karena memberi fleksibilitas dokumen untuk *field* yang berubah *sekaligus* transaksi ACID yang dibutuhkan pola *outbox*, dengan satu teknologi saja. Konsekuensinya, isi `attributes` belum divalidasi tipenya dan belum diindeks.

6 Problem 5 – Fan-out Event Distribution (Pub/Sub)

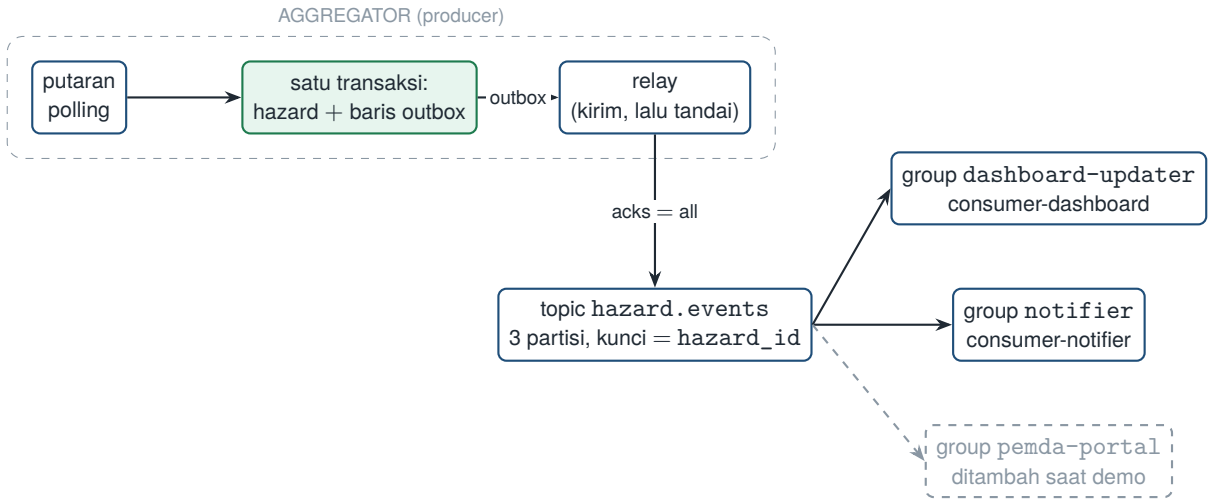
Ringkasan

Aggregator **tidak pernah memanggil *consumer***. Setiap perubahan hazard ditulis ke tabel *outbox* dalam transaksi yang sama dengan hazard-nya, lalu sebuah *relay* mempublikasikannya ke *topic* Kafka `hazard.events`. Setiap *consumer* men-*subscribe* dengan *consumer group* masing-masing. Pengiriman bersifat **at-least-once**, sehingga *consumer* membuang duplikat berdasarkan `event_id`. *Consumer* yang mati tidak menahan *producer*, dan akan menyusul *event* yang terlewat saat hidup kembali.

Status bukti Problem 5

Keempat kriteria dibuktikan oleh satu skrip, `make demo-p5`: *event* lewat *broker*, dua *consumer* independen, *consumer* yang mati lalu menyusul *dengan ID event yang tepat*, dan *consumer* ketiga (`pemda-portal`) yang ditambahkan saat sistem berjalan tanpa menyentuh Aggregator. *Consumer* ketiga ini tidak ada di Compose, dan dinyalakan oleh skrip demo untuk membuktikan bahwa menambah *consumer* tidak membutuhkan perubahan pada *producer*.

6.1 Rancangan solusi dan mekanisme



Gambar 6.1: Alur event. Producer hanya mengenal topic; consumer tidak dikenal producer. Garis putus-putus: consumer yang ditambahkan saat demo tanpa mengubah producer.

6.1.1 Sisi producer: transactional outbox

1. Saat Aggregator menyimpan hazard baru atau versi baru, ia *juga* menulis satu baris `outbox` dalam **transaksi database yang sama**. Keduanya berhasil bersama atau gagal bersama, sehingga tidak ada hazard yang tersimpan tanpa *event*.
2. ***Relay*** (`aggregator/internal/relay`) membaca baris *outbox* yang belum terkirim, mengirimnya ke Kafka berurutan dari yang paling lama, dan baru menandainya terkirim *setelah* Kafka mengakuinya. Jika satu pengiriman gagal, *relay* berhenti di titik itu (untuk menjaga

urutan per hazard), menunggu dengan jeda yang makin panjang, lalu mencoba lagi.

3. Akibatnya, jika Kafka mati, ingest tidak berhenti dan tidak ada *event* yang hilang. *Event* menunggu di *outbox* dan terkirim berurutan setelah *broker* pulih (`TestBrokerOutageDoesNotStopIngestion`).
4. *Producer* Kafka memakai `acks=all` dan mode idempoten (`franz-go`).

Item	Nilai
Topic	`hazard.events` dengan 3 partisi, replikasi 1, dan retensi 7 hari. Dibuat oleh `kafka-init`, dan pembuatan *topic* otomatis dimatikan.
Key pesan	`hazard_id`. Semua versi satu hazard masuk ke satu partisi, sehingga urutannya per hazard terjaga.
Isi pesan	JSON dengan `event_id`, `kind` (`created` atau `updated`), `emitted_at`, `hazard`, dan `schema`. Skemanya ada di `contracts/schemas/kafka-envelope.json`.
Header pesan	`correlation_id` (sama dengan siklus *polling* yang menghasilkan perubahan) dan `event_id`.
Key idempotensi	`event_id = <hazard_id>:<version>`
Consumer group	Satu group per *consumer*: `dashboard-updater` dan `notifier`. Group `pemda-portal` ditambahkan saat demo.

Tabel 6.1: Konfigurasi Kafka dan format pesan.

6.1.2 Sisi consumer

Setiap *consumer* (`services/consumer-hazard`) men-*subscribe topic* dengan group-nya sendiri. Untuk tiap pesan, *consumer* (1) membaca *header* untuk melanjutkan `cid`, (2) membuang pesan yang `event_id`-nya sudah pernah diproses, (3) memproses pesan, lalu (4) menandai *offset* untuk di-*commit*. Karena *offset* ditandai *setelah* pesan diproses, pesan tidak pernah hilang jika *consumer* mati di tengah jalan. Dalam kasus terburuk, pesan itu diproses lagi.

6.1.3 Semantik pengiriman: at-least-once

- *Producer* ke Kafka: *relay* menandai baris terkirim hanya setelah mendapat pengakuan. Jika Aggregator mati *tepat di antara* pengakuan dan penandaan, baris yang sama dikirim ulang. Satu *event* bisa tiba dua kali, tetapi tidak pernah nol kali.
- Kafka ke *consumer*: *offset* di-*commit* setelah pemrosesan, sehingga pesan yang belum selesai diproses akan dikirim ulang setelah *consumer* hidup kembali.
- **Konsekuensinya, *consumer* harus idempotent.** Kelompok menanganinya dengan membuang duplikat berdasarkan `event_id`, yang bersifat deterministik (`hazard_id:version`).

6.2 Alasan keputusan dan trade-off

Tabel berikut membandingkan beberapa cara mendistribusikan *event* dengan pilihan kami.

Pola yang dipakai adalah *publish-subscribe* dengan satu *consumer group* untuk tiap pihak yang berlangganan. Jika dua *consumer* dimasukkan ke group yang sama, Kafka akan *membagi event* di antara mereka (pola antrean kerja), bukan menyalinnya. Karena tiap pihak harus menerima *semua event*, kami memberi tiap pihak group sendiri.

Untuk semantik pengiriman, *at-most-once* ditolak karena *event* peringatan bisa hilang tanpa diketahui, dan itu tidak dapat diterima untuk data kebencanaan. *Exactly-once* ditolak karena menuntut transaksi dari ujung ke ujung dan jauh lebih rumit. *Consumer* yang idempotent menghasilkan

Alternatif	Pertimbangan
Producer memanggil tiap *consumer* langsung (HTTP)	*Consumer* yang lambat menahan *producer*, dan menambah *consumer* baru berarti mengubah kode *producer*. Ini justru masalah yang ingin diselesaikan.
RabbitMQ	Antrean yang matang dan ringan. Pada antrean biasa, pesan yang sudah dikonsumsi dihapus, sehingga *consumer* baru tidak bisa membaca ulang dan setiap *consumer* membutuhkan antrean sendiri yang harus dikelola. (RabbitMQ Streams menutup celah ini, tetapi itu pola lain yang tidak kami telusuri.)
NATS (core)	Sangat ringan dan cepat, tetapi tidak menyimpan pesan. *Consumer* yang mati kehilangan *event*, sehingga tidak memenuhi kriteria menyusul. (JetStream menambahkan penyimpanan, tetapi tidak kami pilih.)
Redis Pub/Sub	Sederhana, tetapi bersifat *fire-and-forget*: *consumer* yang mati tidak menerima apa pun saat kembali.
Apache Kafka (dipilih)	*Log* yang tahan lama. Pesan disimpan sesuai retensi dan tiap *consumer group* punya *offset* sendiri. *Consumer* yang mati menyusul dari *offset* terakhirnya, dan group baru dapat men-*subscribe* tanpa menyentuh *producer*.

Tabel 6.2: Alternatif *broker* dan pola distribusi.

efek yang sama dengan biaya yang jauh lebih kecil.

Konsekuensi memilih Kafka adalah bebannya yang lebih berat dibanding RabbitMQ atau NATS (membutuhkan memori JVM, dengan heap dibatasi 256–512 MB di Compose) dan operasinya yang lebih rumit. Untuk kebutuhan menyusul setelah mati dan menambah *consumer* tanpa mengubah *producer*, harga itu kami nilai layak.

6.3 Batasan yang disadari

- Pembuangan duplikat hanya dilakukan di memori. Kumpulan `event_id` hilang saat *consumer* di-*restart*, dan entri lama tidak pernah dihapus (memori tumbuh seiring jumlah *event*). Setelah *restart*, pesan yang belum di-*commit* bisa diproses lagi. Pada M1 *consumer* hanya mencatat dan menghitung sehingga aman. Untuk aksi nyata (misalnya notifikasi sungguhan), pemrosesan ulang harus aman diulang atau memakai penyimpanan dedup yang tahan lama.
- Retensi 7 hari. *Consumer* yang mati lebih lama dari itu kehilangan *event* yang sudah kedaluwarsa.
- Group baru membaca semua riwayat yang masih tersimpan. Hal ini kami periksa secara terpisah dari skrip demo: group baru memproses 153 *event*, sama dengan jumlah di dashboard. Perilaku ini berguna untuk *consumer* yang perlu mengejar data, tetapi *consumer* yang hanya menginginkan *event* baru harus mengaturnya sendiri.
- Hanya ada satu *broker* dengan replikasi 1. Jika disk *broker* hilang, *event* yang belum dibaca ikut hilang. Replikasi dan penanganan kegagalan bersamaan menjadi bahan untuk M2.
- Urutan hanya terjamin per hazard (satu key, satu partisi), bukan urutan global lintas hazard.
- Pesan yang rusak dilewati dan dicatat di *log* (`consumer_bad_message`). Belum ada *dead-letter queue*.
- *Consumer* pada M1 bersifat tiruan: dashboard dan notifier hanya mencatat dan menghitung *event*, belum menggerakkan tampilan atau mengirim notifikasi sungguhan.
- Hanya ada satu *relay* untuk satu Aggregator. Jika Aggregator dijalankan dalam beberapa *instance* (M2), pembacaan *outbox* perlu dikunci agar *event* tidak terkirim ganda.

- *Consumer* untuk portal pemerintah daerah (`pemda-portal`) hanya ada saat demo dan tidak termasuk dalam Compose sehari-hari.

6.4 Bukti penyelesaian

Bukti berasal dari `make demo-p5` (transkrip: `docs/evidence/p5/latest.log`) dan dari tes otomatis di `aggregator/internal/relay`.

Kriteria pada spesifikasi	Bukti	Status
Event dipublikasikan ke *broker*, bukan langsung ke *consumer*	Alur *outbox* dan *relay*. Skrip demo menunggu *log* `outbox_published` untuk ID *event* yang baru dibuat, yang menandakan Kafka sudah mengakuinya. Tes ada di `relay_test.go` (daftar di bawah tabel). Aggregator tidak memiliki alamat *consumer* mana pun di konfigurasinya.	**Lulus**
Dua *consumer* independen menerima *event*, dan *producer* tidak mengenal mereka	Langkah B: satu *event* BMKG baru dibangkitkan, dan kedua *consumer* (group berbeda) sama-sama mencatat ID yang sama (`bmkg-EQ-9b73f8-0061:1`) di lognya.	**Lulus**
Satu *consumer* dihentikan: *producer* tidak tertahan dan *consumer* lain normal; setelah hidup lagi, apa yang terjadi pada *event* yang terlewat	Langkah C: `consumer-notifier` dihentikan, Aggregator tetap menyimpan dan mempublikasikan *event* baru (`bmkg-EQ-9b73f8-0063:1`), dan dashboard tetap menerimanya. Langkah D: notifier dinyalakan lagi dan **menyusul** *event* yang tepat itu. Skrip memeriksa ID *event* di *log* notifier, bukan sekadar kenaikan angka. *Event* tidak hilang karena Kafka menyimpannya dan *offset* group dicatat di *broker*.	**Lulus**
Menambah *consumer* baru pada *topic* yang sudah ada, tanpa mengubah kode *producer*	Langkah E: *container* baru dengan `KAFKA_GROUP=pemda-portal` dinyalakan lewat `docker compose run` saat sistem berjalan. *Event* berikutnya (`bmkg-EQ-9b73f8-0065:1`) diterima oleh ketiga group. Skrip memeriksa bahwa ID *container* dan waktu start Aggregator tidak berubah.	**Lulus**

Tabel 6.3: Kriteria Problem 5 dan buktinya.

Cuplikan transkrip (`make demo-p5`)

```
[1] A. Both consumers are healthy and subscribed
    PASS consumer-dashboard /health        PASS consumer-notifier /health
[2] B. New hazard event is published and received by BOTH consumers
    generated BMKG event EQ-9b73f8-0061 - waiting for Kafka fan-out
    PASS dashboard processed=126 notifier processed=23
[3] C. Notifier stopped - Aggregator still ingests; dashboard still receives events
    PASS aggregator ingested bmkg-EQ-9b73f8-0063 while notifier is down
    PASS dashboard still consuming (producer not blocked by dead consumer)
[4] D. Notifier restarts and catches up
    PASS notifier caught up with the exact offline event bmkg-EQ-9b73f8-0063:1
[5] E. Add a new consumer group to the same topic without touching the producer
    PASS dashboard, notifier, and new pemda group received bmkg-EQ-9b73f8-0065:1;
         producer was unchanged
    PASS P5 pub/sub demo complete
```

Angka `processed` berbeda antara dashboard dan notifier karena skrip me-*restart* notifier, dan penghitungnya disimpan di memori sehingga mulai dari nol lagi. Karena itu skrip membuktikan ID *event*, bukan jumlahnya.

Tes otomatis yang terkait (`relay_test.go`):

- `TestFlushPublishesInOrderWithHeadersAndMarksDelivered`: urutan, *header*, dan penandaan terkirim.
- `TestBrokerOutageDoesNotStopIngestionOrLoseEvents`: *broker* mati tidak menghentikan ingest dan tidak menghilangkan *event*.
- `TestPartialFailureKeepsOrderAndResumes`: kegagalan di tengah batch tetap menjaga urutan, dan pengiriman dilanjutkan.
- `TestRunDeliversNewEntriesUntilCancelled`: *relay* terus mengirim entri baru sampai dihentikan.

6.5 Jawaban

Broker dan pola yang dipilih

Broker yang dipilih adalah **Apache Kafka** (mode KRaft, satu *broker*) dengan pola *publish-subscribe*: satu *topic* `hazard.events` dan satu *consumer group* untuk tiap pihak yang berlangganan. Kafka dipilih karena pesannya disimpan, sehingga *consumer* yang mati dapat menyusul dan group baru dapat ditambahkan tanpa mengubah *producer*. *Producer* menulis lewat *transactional outbox* agar tidak ada *event* yang hilang saat *broker* sedang mati.

Semantik pengiriman dan konsekuensi bagi *consumer*

Semantik yang berlaku adalah **at-least-once**. *Producer* menunggu pengakuan penuh (`acks=all`, idempotent), *relay* menandai terkirim setelah pengakuan, dan *consumer* meng-*commit offset* setelah memproses. *Event* tidak hilang, tetapi dapat tiba lebih dari sekali. Karena itu ***consumer* harus idempotent**. Kelompok menanganinya dengan membuang duplikat berdasarkan `event_id = hazard_id:version`, yang bersifat deterministik. Batasannya, kumpulan dedup berada di memori dan hilang saat *restart*, sehingga aksi sungguhan pada *consumer* di kemudian hari harus aman diulang.

A Penelusuran Klaim ke Bukti

A.1 Ketentuan implementasi lintas problem

Ketentuan pada spesifikasi	Cara memeriksa	Kode atau bukti
Komunikasi lewat jaringan, tanpa berbagi library business-logic	`pkg/obs` hanya utilitas teknis; tiap *service* punya `go.mod` sendiri	`pkg/obs`, `services/*/go.mod`
Satu *storage* satu pemilik, dan Canonical Store hanya diakses Aggregator	`make demo-isolation`	`docs/evidence/p4/isolation.log`
Satu *container* per *service* dan satu perintah orkestrasi	`make reset` atau `docker compose up --build`	`services/*/Dockerfile`, `docker-compose.yml`
Kredensial satu lembaga tidak berlaku di lembaga lain	`make demo-mock-auth`	`docs/evidence/p3/mock-auth.log`
Tidak ada kredensial di kode atau *commit*, dan `.env.example` tersedia	`make secrets`	`.env.example`, `.gitignore`, `.gitleaks.toml`
`/health`, *log* terstruktur dengan `cid`, dan latensi panggilan keluar	Langkah 8 demo Problem 1 dan `cid-trace.log` pada Problem 2	`pkg/obs`
Polling dari BNPB, interval dapat diatur, sedikitnya satu *event* per 10 detik, dan seed 20 *record*	Tes pada *mock* dan demo Problem 1	`services/*-mock/internal/*/generator.go`
Kegagalan benar-benar dipicu saat demo	`make demo-p2-aggregator`, `make demo-p2`	`POST /admin/outage`, `POST /admin/delay`

Tabel A.1: Ketentuan implementasi dan cara memeriksanya.

A.2 Matriks kriteria, perintah, dan bukti

Problem	Perintah	Berkas bukti	Status
P1	`make demo-p1`	`docs/evidence/p1/latest.log`	**Lulus**
P2 (ingest)	`make demo-p2-aggregator`	`docs/evidence/p2/aggregator-latest.log`	**Lulus**
P2 (layanan dan *load test*)	`make demo-p2`	`docs/evidence/p2/latest.log`, `k6-summary.json`, `k6-overload-summary.json`, `cid-trace.log`	**Lulus**
P3 (kredensial *mock*)	`make demo-mock-auth`	`docs/evidence/p3/mock-auth.log`	**Lulus**
P3 (klien downstream)	`make demo-p3`	`docs/evidence/p3/latest.log`	**Lulus**

Problem	Perintah	Berkas bukti	Status
P3 (tidak ada kredensial ter-*commit*)	`make secrets`	keluaran `scripts/check-secrets.sh`	**Lulus**
P4 (isolasi *storage*)	`make demo-isolation`	`docs/evidence/p4/isolation.log`	**Lulus**
P4 (JSONB, *record* berdampingan)	`make demo-p4` (langkah B–D)	`docs/evidence/p4/latest.log`	**Lulus**
P4 (stop, *rebuild*, dan jalankan ulang satu *container*)	`make demo-p4` (langkah A)	`docs/evidence/p4/latest.log`	**Lulus**
P5 (fan-out, *consumer* mati, lalu menyusul)	`make demo-p5`	`docs/evidence/p5/latest.log`	**Lulus**
P5 (*consumer* baru tanpa mengubah *producer*)	`make demo-p5` (langkah E)	`docs/evidence/p5/latest.log`	**Lulus**

A.3 Perintah untuk pemeriksaan penuh

```
make lint && make test-race                                    # kualitas kode dan tes (dengan
    race detector)
make reset && make demo-p1 demo-p2-aggregator demo-mock-auth demo-isolation
make demo-p4 demo-p5                                           # rebuild satu service, field baru
     tanpa migrasi, consumer baru
make demo-p2 demo-p3                                           # termasuk load test dan menunggu
    token kedaluwarsa
make secrets                                                   # sebelum tag milestone-1
```

B Lampiran Surat Pernyataan Penggunaan AI

Surat pernyataan penggunaan AI yang sudah diisi dan ditandatangani ketiga anggota terlampir pada dua halaman berikut.

SURAT PERNYATAAN PENGGUNAAN AI

Dengan ini, kami:

Nama/NIM:
1. Andi Farhan Hidayat / 13523128
2. Muhammad Rizain Firdaus / 13523164
3. Muhammad Kinan Arkansyaddad / 13523152

Fakultas/Sekolah : STEI
Kode Mata Kuliah : IF4031
Nama Mata Kuliah : Arsitektur Aplikasi Terdistribusi
Judul Tugas/Proposal : Milestone 1 Tugas Besar IF4031 Arsitektur Aplikasi Terdistribusi

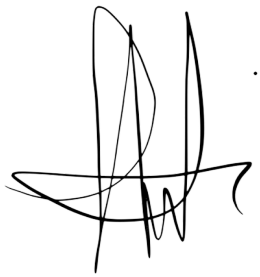
Menyatakan bahwa kami menggunakan AI dalam pengerjaan maupun penyusunan luaran tugas pada mata kuliah yang tertulis di atas.

Jika Ya, maka alat AI/kecerdasan buatan yang digunakan adalah:

Lingkup Pekerjaan	Digunakan?	Tingkat Penggunaan
Pemeriksaan Ejaan: menggunakan tools seperti Grammarly, Paperpal, Deepl, Quillbot, Grammarbot, LanguageTool, ProWritingAid, ChatGPT, Google Gemini, atau sejenisnya.	Tidak	-
Pembuatan Teks: menggunakan tools seperti ChatGPT, Gemini, Paperpal, Copilot, GrammarlyGO, WordAI, WriteSonic, Jasper, Jenni AI, atau sejenisnya.	Ya	Sedang
Bantuan Diskusi dalam Penyusunan Konten: menggunakan tools seperti ChatGPT, Gemini, Copilot, Perplexity, Jenni AI, Zoom-Companion, atau sejenisnya.	Ya	Sedang
Pembuatan Gambar, Video, dan/atau Grafik: menggunakan tools seperti Craiyon, DALL-E, Midjourney, Stable Diffusion, Microsoft Designer, Gemini, Canva AI, atau sejenisnya.	Tidak	-

Kami juga menyatakan bahwa setiap penggunaan AI/kecerdasan buatan yang dilakukan pada mata kuliah tersebut telah dinyatakan di dalam surat ini.

Bandung 10 Oktober 2026







Andi Farhan Hidayat, 13523128	Muhammad Rizain Firdaus, 13523164	Muhammad Kinan Arkansyaddad, 13523152